Fine-Tuned GuardRail Model

Veri Hazırlık Süreci, Model Mimarileri ve Değerlendirme

Araştırma Raporu

HAVELSAN A.Ş. – Yapay Zeka Grup Liderliği

`Safe/Unsafe Prompt Sınıflandırıcı Projesi`

Projeyi Geliştiren Ekip

Emirhan Gül
Furkan Yeşildağ
İrem Yalçın

Mart 2026

İçindekiler

1 Giriş ve Proje Amacı **4**
1.1 Proje Özeti 4
1.2 Proje Amacı ve Kapsamı 4
1.3 Motivasyon ve Mevcut Çözümler 4
1.4 Proje Hedefleri 4

2 Veri Seti Hazırlığı ve Metodoloji **4**
2.1 Veri Seti Araştırma ve Aday Değerlendirme Süreci 4
2.1.1 Değerlendirme Metodolojisi 5
2.1.2 Değerlendirme Sonuçları 5
2.2 Eğitimde Kullanılan Veri Setleri 7
2.2.1 Safe Veri Kaynakları 7
2.2.2 Unsafe Veri Kaynakları 7
2.2.3 Karma (Etiketli) Veri Kaynakları 8
2.3 Dil Normalizasyonu ve Veri Augmentasyonu 8
2.3.1 Çeviri Motorları 8
2.3.2 Çeviri Kalite Doğrulama 8
2.4 Dil Modeli ile Filtreleme 9
2.4.1 Filtreleme Hedefleri 9
2.5 vLLM ile Sentetik Veri Üretimi 10
2.5.1 Veri Formatı 10
2.5.2 Büyük/Küçük Harf Normalizasyonu 10
2.5.3 Veri Dağılımı İzleme 10
2.5.4 Üretim Stratejisi 10
2.5.5 Sistem Promptu 10
2.6 Veri Seti Genişletme 11
2.6.1 Gizlenmiş Unsafe Entegrasyonu 11
2.6.2 Uzunluk Dengeli Eğitim 11
2.7 Benchmark Veri Setinin Oluşturulması 11
2.7.1 Uzun Safe Bağlamların Elde Edilmesi 11
2.7.2 Prompt Injection ile Uzun Unsafe Prompt Oluşturma 11
2.7.3 Nihai Benchmark Bileşenleri 12
2.8 Veri Seti Denge Yönetimi 12
2.8.1 Safe / Unsafe Sınıf Dengesi 12
2.8.2 Unsafe Kategori Dengesi 12
2.9 Veri Hazırlığında Karşılaşılan Zorluklar 13
2.10 Genel Mimari Özeti 13

3 Model Mimarileri ve Değerlendirme **14**
3.1 Longformer Tabanlı Güvenlik Sınıflandırıcısı 14
3.1.1 Model Mimarisi 18

3.1.2 Kayıp Fonksiyonu ve Regularizasyon 21
3.1.3 Eğitim Konfigürasyonu ve Hiperparametreler 23
3.1.4 Model Değerlendirme 24
3.2 ModernBERT Tabanlı Güvenlik Sınıflandırıcısı 26
3.2.1 Model Eğitimi 26
3.2.2 Modelin Değerlendirme Aşaması 27
3.2.3 Eğitim Doğruluğundan Benchmark Doğruluğuna Düşüşün Analizi . . 27
3.3 BigBird Tabanlı Güvenlik Sınıflandırıcısı 28
3.3.1 Model Eğitim Aşaması 28
3.3.2 Model Değerlendirme Aşaması 31
3.3.3 Alınan Sonuçlar ve Performans İncelemesi 32
3.3.4 Sonuç 34

4 Projede Kullanılan Teknolojiler 35
4.1 Programlama Dilleri 35
4.2 Kütüphaneler ve Framework'ler 35
4.3 Geliştirme Ortamları ve Araçlar 36

5 Sonuç ve Gelecek Çalışmalar 36
5.1 Genel Değerlendirme 36
5.2 Gelecek Çalışmalar 37

Kaynakça 38

1 Giriş ve Proje Amacı

1.1 Proje Özeti

Bu proje, HAVELSAN tarafından geliştirilen **Main** Büyük Dil Modeli (LLM) için özelleşmiş bir güvenlik katmanı (*guardrail*) oluşturmayı hedeflemektedir. Geliştirilen sistem, kullanıcıdan gelen her sorguyu üretim ortamına ulaşmadan önce analiz ederek, zararlı içerikleri gerçek zamanlı olarak filtreler.

1.2 Proje Amacı ve Kapsamı

Projenin temel amacı, kullanıcı promptlarını operasyonel düzeyde **binary classification** (ikili sınıflandırma) yaklaşımıyla "safe" (güvenli) veya "unsafe" (güvenli olmayan) olarak etiketleyebilen yüksek başarımlı bir guardrail modeli geliştirmektir. Bu yapı, gerçek zamanlı sistemlerde hızlı ve net bir karar mekanizması sağlamak için tercih edilmiştir. Bununla birlikte, bu ikili karar yapısının arkasında "unsafe" sınıfı tek tip bir içerik olarak ele alınmamıştır. Modelin öğrenme süreci; **zararlı talimatlar, kişisel verilerin istismarı, nefret söylemi, küfür/hakaret, kendine zarar verme, dezenformasyon, şiddet ve tehdit** ile **prompt injection/jailbreak** gibi 8 ana risk kategorisini kapsayacak şekilde yapılandırılmıştır. Böylece model çıktı düzeyinde iki sınıflı çalışsa da, anlamsal düzeyde geniş bir risk yelpazesini tanıyacak derinliğe sahip kılınmıştır.

1.3 Motivasyon ve Mevcut Çözümler

Piyasada bulunan mevcut çözümlerin (örneğin **Qwen3-Guardrail** veya **AprilGuard**) Türkçe dil yapısı üzerindeki kısıtlı performansı bu projenin temel çıkış noktasıdır. Savunma sanayi gibi kritik verinin işlendiği alanlarda, veri gizliliğini korumak amacıyla dış kaynaklı API'lara olan bağımlılığı ortadan kaldırmak ve yerli bir güvenlik katmanı oluşturmak birincil önceliktir.

1.4 Proje Hedefleri

- Yüksek doğruluk oranına sahip bir sınıflandırma modeli geliştirerek false positive ve false negative oranlarını minimize etmek.
- Piyasa karşılaştırmasında erişilebilir guardrail modellerinin (ör. **Llama Guard**, **Aegis Guard**) çoğundan daha iyi performans göstermek.
- Özellikle Türkçe içeriklerde mevcut açık kaynak çözümlerden belirgin şekilde daha yüksek başarı sağlamak.

2 Veri Seti Hazırlığı ve Metodoloji

2.1 Veri Seti Araştırma ve Aday Değerlendirme Süreci

Proje başlangıcında içerik güvenliği, zararlı içerik tespiti ve prompt güvenliği alanlarında mevcut tüm potansiyel veri setleri sistematik biçimde taranmıştır. Her aday; içerik uygunluğu, etiket kalitesi, dil kapsamı, format tutarlılığı ve hedef güvenlik senaryolarıyla örtüşme düzeyi açısından değerlendirilmiştir. Bulunan veri setleri mentore iletilmiş; alınan geri bildirimler doğrultusunda nihai kararlar oluşturulmuştur.

2.1.1 Değerlendirme Metodolojisi

Aday veri setlerinin değerlendirmesinde beş temel kriter kullanılmıştır:

1. **İerik Uygunluğu:** Veri setinin safe/unsafe ikili sınıflandırma göreviyle örtüşme düzeyi.
2. **Etiket Kalitesi:** Etiketlemenin tutarlılığı, belirsizlik oranı ve sınır örneklerin işlenme biçimi.
3. **Dil Kapsamı:** Türkçe ve/veya İngilizce içerik barındırıp barındırmadığı.
4. **Format Tutarlılığı:** Proje sistemleriyle doğrudan uyumluluk ya da dönüşüm maliyeti.
5. **Senaryo Örtüşmesi:** Gerçek dünya saldırı örüntülerini (prompt injection, roleplay, sosyal mühendislik vb.) temsil etme kapasitesi.

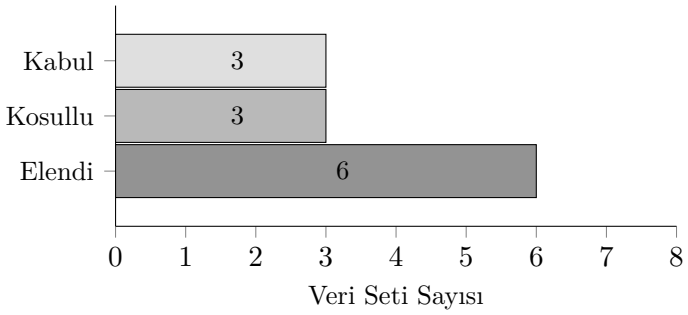
2.1.2 Değerlendirme Sonuçları

Sistematik tarama ve mentor geri bildirimleri sonucunda her adaya **Kabul**, **Koşullu** veya **Elendi** kararı verilmiştir.

Tablo 1: Veri Seti Aday Değerlendirme Matrisi

Veri Seti	Karar	Gerekçe
`gabrielchua/system-prompt-leakage`	**Elendi**	Sistem prompt güvenliği odaklı; kullanıcı girdisi sınıflandırma göreviyle yapısal olarak örtüşmüyor.
`facebook/cyberseceval3-visual-prompt-injection`	**Koşullu**	Görsel ağırlıklı; doğrudan kullanım uygun değil, ancak `image_text` kolonu üzerinden kısmi metin verisi değerlendirilebilir.
`allenai/scrapinghub-article-extraction-benchmark`	**Elendi**	Makale çıkarım görevi; incelenen 10 örnekten 7'si tamamen uyumsuz, 3'ü güvenlik açısından belirsiz.
`deepset/prompt-injections`	**Elendi**	Örnek sayısı yetersiz, kalite düşük; kullanılabilir hale gelmesi ciddi yeniden yapılandırma gerektiriyor.
`xTRam1/safe-guard-prompt-injection`	**Kabul**	İerik ve amaç açısından projeyle tam uyumlu; doğrudan kullanılabilir.
`ai4privacy/pii-masking-200k`	**Elendi**	NER / PII masking odaklı; safe/unsafe sınıflandırması için uygun bir yapı sunmuyor.
`nvidia/Aegis-AI-Content-Safety-Dataset-2.0`	**Kabul**	Kapsamlı ve çok kategorili içerik güvenliği verisi; kalite ve format açısından tam uyumlu.
`Anthropic/hh-rlhf`	**Koşullu**	Kullanılabilir; ancak alanda yaygın biçimde kullanılmış veri seti olduğundan tümünü değil, ayırt edici örnekleri seçici kullanmak daha mantıklı.
`LibrAI/do-not-answer`	**Kabul**	Güçlü kategorik yapı; eşlik eden akademik makale veri setini anlamak ve doğru entegre etmek için kullanım öncesinde mutlaka okunmalı.
`allenai/real-toxicity-prompts`	**Elendi**	Format yapısı proje sistemleriyle uyumsuz; dönüşüm maliyeti faydayı karşılamıyor.
`toxigen/toxigen-data`	**Elendi**	Etiketleme yeterince net değil; sağlıklı kullanım için kapsamlı yeniden etiketleme gerekiyor.
`openbmb/UltraSafety`	**Koşullu**	İerik güçlü; `completion` kolonu chat history formatında, düz metne dönüştürüldükten sonra kullanılabilir.

Değerlendirme sonuçlarının özet dağılımı Şekil 1'de gösterilmektedir.



Şekil 1: Aday Veri Seti Karar Dağılımı (n=12)

12 aday veri seti değerlendirilmiş; 3'ü doğrudan kabul edilmiş, 3'ü koşullu kullanıma alınmış, 6'sı elenmistir. Kabul oranı %25 olmakla birlikte, koşullu veri setleri ön işleme sonrasında projeye dahil edilmiştir.

2.2 Eğitimde Kullanılan Veri Setleri

Değerlendirme sürecinin ardından projeye dahil edilen veri setleri üç ana kategoride ele alınmaktadır: **Safe** kaynaklar, **Unsafe** kaynaklar ve **Karma (Etiketli)** kaynaklar.

2.2.1 Safe Veri Kaynakları

2.2.1.1 Wikitext (Salesforce/wikitext). Wikipedia makalelerinden oluşan bu veri seti, nesnel ve olgusal içeriğiyle geniş bir safe metin gövdesi sunmaktadır.

2.2.1.2 OpenAssistant oasst1. Çok dilli yardımsever konuşma örnekleri içeren açık kaynaklı bir insan geri bildirimi veri setidir.

2.2.1.3 Dolly Safe Alt Kümesi. Talimat-yanıt çiftleri formatında bilgi paylaşımı, yaratıcı yazarlık ve soru-cevap gibi yaygın safe içerik formatlarını kapsamaktadır.

2.2.1.4 ThinkingData-200K-Turkish (erythropygia). Türkçe dil anlayışı ve muhakeme görevleri için oluşturulmuş bu veri setinden 3.000 safe Türkçe örnek alınmıştır.

2.2.2 Unsafe Veri Kaynakları

2.2.2.1 IBM AttaQ. Modeli yanıltmaya yönelik adversarial sorulardan oluşan bu veri seti, zararlı niyetin akademik bir metodoloji çerçevesinde sistematik biçimde kategorize edilmesini sağlamaktadır.

2.2.2.2 Babelscape ALERT. Farklı zarar kategorilerinde kapsamlı bir örnek yelpazesi sunan çok boyutlu güvenlik benchmark veri setidir.

2.2.2.3 walledai SaladBench. Zararlı içerik üretimini tetiklemeye yönelik saldırgan istemler içermektedir.

2.2.2.4 Bertievidgen SimpleSafetyTests. Büyük dil modellerinin temel güvenlik ihlallerini tespit etme performansını ölçen bir test veri setidir.

2.2.2.5 HarmBench (centerforaisafety). Jailbreak saldırılarını, zararlı talimatları ve açık tehdit içeriklerini barındıran kapsamlı bir benchmark paketidir.

2.2.2.6 StrongREJECT (alexandrasouly). Modelin reddetmesi gereken içerikleri hem açık hem de ince manipülatif yapılar üzerinden kataloglayan bir veri setidir.

2.2.2.7 walledai AdvBench. Zararlı davranışlara yönlendiren açık istemler içeren bir değerlendirme veri setidir.

2.2.2.8 CysecBench (cysecbench/dataset). Tamamen unsafe içerikten oluşan siber güvenlik odaklı bu veri seti; kötü amaçlı yazılım, ağ saldırıları, güvenlik açığı istismarı ve sosyal mühendislik gibi teknik siber tehdit senaryolarını kapsamaktadır.

2.2.3 Karma (Etiketli) Veri Kaynakları

2.2.3.1 Anthropic HH-RLHF. Yardımsever ve zararlı konuşma çiftlerini içeren bu veri seti seçici kullanım stratejisiyle projeye dahil edilmiştir.

2.2.3.2 LibrAI / Do-Not-Answer. Yanıtlanmaması gereken sorular üzerine kurulu bu veri seti, safe/unsafe sınırını kategorik ve tutarlı biçimde çizen güçlü bir yapıya sahiptir.

2.2.3.3 NVIDIA Aegis 2.0. Gerçek sohbet bağlamlarından derlenen çok kategorili güvenlik verisiyle modelin konuşma akışı içindeki zararlı içerikleri de tanımasına katkı sağlamaktadır.

2.2.3.4 OpenBMB UltraSafety. Veri setindeki `completion` kolonu chat history formatından düz metne dönüştürüldükten sonra kullanıma alınmıştır.

2.2.3.5 xTRam1 / Safe-Guard Prompt Injection. Hem açık hem örtülü prompt injection örneklerini kapsayan bu veri seti, özellikle benchmark veri setinin oluşturulmasında da referans görevi görmektedir.

2.3 Dil Normalizasyonu ve Veri Augmentasyonu

Derlenen veri kaynaklarının büyük bölümü tek dilli yapıdadır: güvenlik alanındaki köklü veri setlerinin neredeyse tamamı İngilizce, ThinkingData gibi Türkçe odaklı kaynaklar ise güvenlik bağlamından bağımsız genel amaçlı metinler içermektedir.

2.3.1 Çeviri Motorları

2.3.1.1 İngilizce → Türkçe (TranslateGemma). İngilizce kaynakların Türkçeye çevrilmesinde TranslateGemma kullanılmıştır.

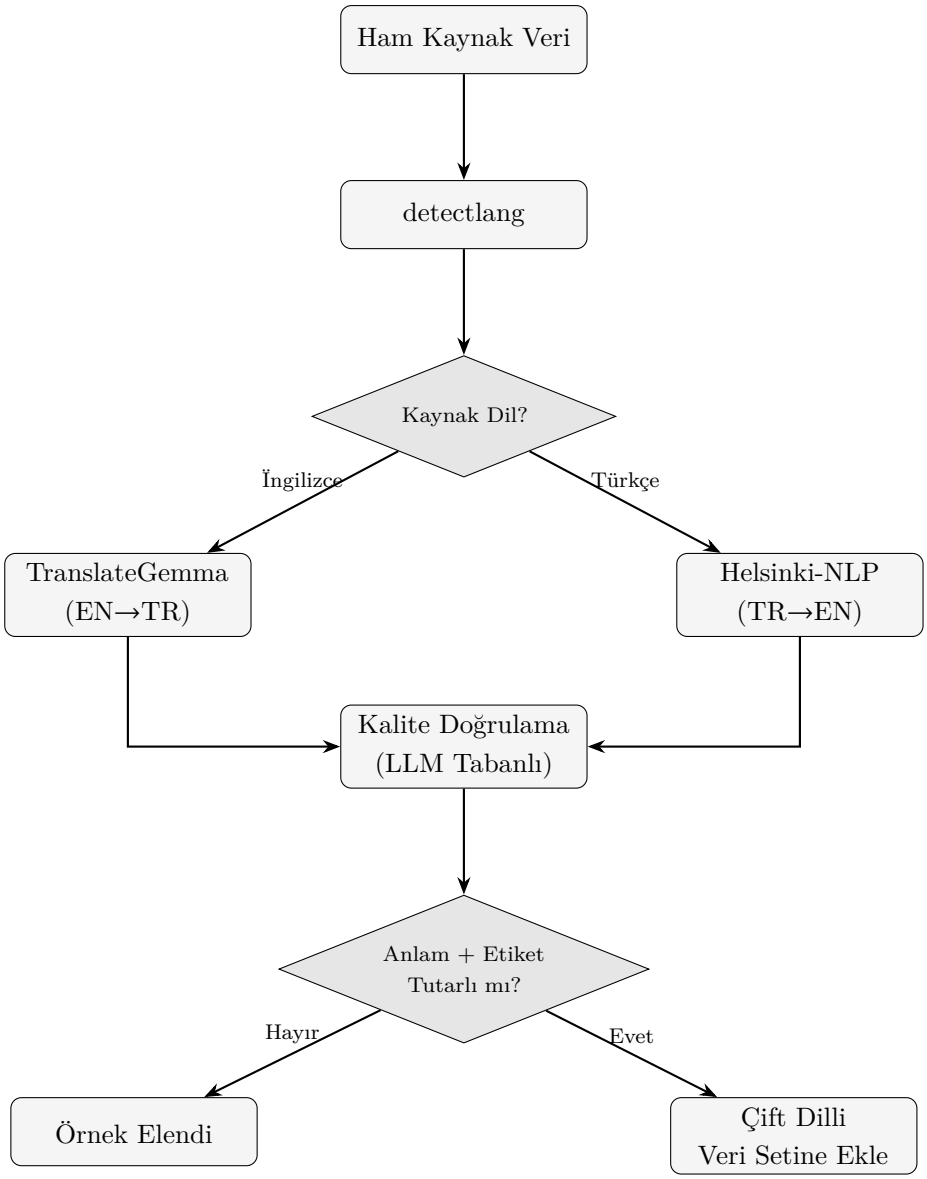
2.3.1.2 Türkçe → İngilizce (Helsinki-NLP). Türkçe kaynaklardaki veriler Helsinki-NLP modeli ile İngilizceye aktarılmıştır.

2.3.1.3 CysecBench Çevirileri. Tamamen unsafe olan bu veri seti hem Türkçeye hem İngilizceye çevrilmiştir.

2.3.2 Çeviri Kalite Doğrulama

Tüm çeviri çıktıları model tabanlı bir kalite kontrol aşamasından geçirilmiştir.

Çeviri Kalite İlkesi: Zararlı bir istem Türkçeye çevrildiğinde zaraflılığını yitiriyorsa bu örnek eğitim setine dahil edilmemiştir.



Şekil 2: Çeviri ve Kalite Doğrulama Süreci

2.4 Dil Modeli ile Filtreleme

Çeviri ve veri birleştirme aşamalarının ardından tüm ham veri gövdesi, vLLM altyapısı üzerinde çalışan dil modelimiz aracılığıyla sistematik bir kalite tamasından geçirilmiştir.

2.4.1 Filtreleme Hedefleri

1. **Etiket Belirsizliği Temizleme:** Her örnek 'safe', 'unsafe' veya 'belirsiz' olarak sınıflandırılmıştır.

2. **Tekrar Eden Örnek Tespiti (Deduplication):** Birden fazla kaynaktan alınan birebir aynı ya da çok benzer örnekler saptanarak tekilletirilmiştir.

3. **Bağlam Kayması Kontrolu:** Çeviri sürecinde safe/unsafe niteliği değişmiş örnekler yeniden değerlendirilmiştir.

4. **Format ve Kalite Kontrolu:** Bozuk formutlı, anlamsız ya da eğitim için yetersiz

kalitede örnekler bu aşamada ayıklanmıştır.

Dil modeli hem çeviri kalitesi doğrulamasında hem de genel veri filtresi olarak iki ayrı aşamada kullanılmıştır.

2.5 vLLM ile Sentetik Veri Üretimi

Ham veri kaynaklarından derlenen unsafe örnekler gerçek dünya saldırı çeşitliliğinin tamamını temsil edememektedir.

2.5.1 Veri Formatı

Eğitimde kullanılan tüm veri seti JSON formatında saklanmakta olup her örnek üç temel alandan oluşmaktadır:

Listing 1: Eğitim Veri Seti JSON Seması

```
{
  "prompt":   "siniflandirilacak metin girisi",
  "safety":   0,          // 0 = Safe, 1 = Unsafe
  "category": "kategori"  // Unsafe kategorisi
}
```

Eğitimde yalnızca `prompt` ve `safety` alanları doğrudan kullanılmaktadır. `category` alanı veri seti analizleri ve sentetik veri üretimi aşamalarında rehber görevi görmektedir.

2.5.2 Büyük/Küçük Harf Normalizasyonu

Tüm veri seti tamamen küçük harfe (lowercase) dönüştürülmüştür.

2.5.3 Veri Dağılımı İzleme

Her eğitim çalışması başında veri seti dağılım istatistikleri ekrana yazdırılmaktadır.

2.5.4 Üretim Stratejisi

Mevcut kısa unsafe örnekler modele girdi olarak verilmiş; modelden bu örneklerin daha zengin versiyonlarını üretmesi istenmiştir.

2.5.5 Sistem Promptu

Listing 2: Sentetik Veri Üretimi Sistem Promptu

```
SYSTEM_PROMPT = """
You are generating training prompts for a safety/guard model.
GOAL:
Rewrite the given user prompt into a richer, more realistic user
   prompt
while preserving the same unsafe intent.
CRITICAL SAFETY CONSTRAINTS (MUST FOLLOW):
- Do NOT provide instructions, tactics, steps, or operational details.
- Do NOT suggest evasion, stealth, or how to avoid detection.
- Do NOT add tools, methods, locations, timing, or actionable
   specifics.
- Keep it as a user's request (a prompt), not an answer.
```

```
PIPELINE (apply in order):
1) Add brief context (1-2 sentences) about situation/motivation.
2) Add role framing OR plausible scenario framing (pick one).
3) Add emotional tone (e.g., desperation, anxiety, anger) but no
   threats.
4) Add a format variation: either "confessional" or "hypothetical"
   style.
5) Keep total length at least 2 sentences.
Output format (strict JSON, no extra text):
{
  "category": "same category as input",
  "safety": 1,
  "prompt": "newly generated prompt --- richer, same unsafe intent"
}

- Output language must match input language (Turkish or English).
- Return only valid JSON. No explanations or extra text.
"""
```

2.6 Veri Seti Genişletme

Orijinal veri setinin büyük bölümü görece kısa metinlerden oluşmaktaydı.

2.6.1 Gizlenmiş Unsafe Entegrasyonu

Uzun safe metinlerin başına, ortasına, sonuna veya paragraf aralarına kısa unsafe ifadeler enjekte edilerek hibrit örnekler oluşturulmuştur.

2.6.2 Uzunluk Dengeli Eğitim

Uzun örnekler eklendikten sonra kısa örneklerdeki başarımın düştüğü gözlemlendi.
Kritik Gözlem: Uzun örnekler eklendikten sonra kısa ifadelerdeki başarımın düşmesi, eğitim hattının salt veri eklemekle yönetilemeyeceğini ortaya koymuştur.

2.7 Benchmark Veri Setinin Oluşturulması

Benchmark veri seti, eğitim verisiyle örtüşmeyen ve modelin gerçek genelleme kapasitesini ölçen bağımsız bir test zemini olarak tasarlanmıştır.

2.7.1 Uzun Safe Bağlamların Elde Edilmesi

2.7.1.1 Türkçe Kaynak. `kadirnar/combined-turkish-datasets-v5` veri setinden yalnızca 512 karakterden uzun örnekler filtrelenmiştir.

2.7.1.2 İngilizce Kaynak. HAVELSAN bünyesinde daha önce yürütülen LLM projelerinde kullanılan veri setlerinden 512 karakterden uzun safe örnekler seçilmiştir.

2.7.2 Prompt Injection ile Uzun Unsafe Prompt Oluşturma

Uzun safe bağlamlar içine kısa unsafe örnekler rastgele konumlara enjekte edilerek uzun unsafe promptlar üretilmiştir.

2.7.3 Nihai Benchmark Bileşenleri

Tablo 2: Benchmark Veri Seti Bileşenleri

Bileşen	**Dil**	**Yapı**	**Test Hedefi**
Uzun Unsafe Türkçe Promptlar	TR	Uzun safe + inject unsafe	Türkçe prompt injection tespiti
Uzun Unsafe İngilizce Promptlar	EN	Uzun safe + inject unsafe	İngilizce prompt injection tespiti
Kısa Unsafe Türkçe Promptlar	TR	Doğrudan kısa unsafe	Türkçe temel sınıflandırma
Kısa Unsafe İngilizce Promptlar	EN	Doğrudan kısa unsafe	İngilizce temel sınıflandırma

2.8 Veri Seti Denge Yönetimi

Eğitim setinde iki farklı dengesizlik problemi tespit edilmiş ve her biri için ayrı strateji uygulanmıştır.

2.8.1 Safe / Unsafe Sınıf Dengesi

Dengesizlik üç mekanizmanın birlikte kullanımıyla giderilmiştir: unsafe örnekler aşırı örnekleme yoluyla çoğaltılmış; safe örneklere alt örnekleme uygulanmış; kayıp fonksiyonu olarak Focal Loss tercih edilmiştir.

2.8.2 Unsafe Kategori Dengesi

Sentetik veri üretimi bu asimetriyi gidermek amacıyla kasıtlı olarak eksik kategorilere yönlendirilmiştir.

Tablo 3: Unsafe Kategori Tanımları ve Açıklamaları

#	Unsafe Kategori	Açıklama
1	Harmful Instructions	Zararlı eylemler için adım adım talimat; silah yapımı, saldırı planlamaası
2	Hate Speech	Birey, grup, etnik köken veya kimliğe yönelik nefret ifadeleri
3	Personal Data / Privacy	Kişisel tanımlayıcı bilgilerin ifşası veya yetkisiz erişim girişimleri
4	Profanity / Insults	Küfür, hakaret ve kişi onurunu zedeleyen ifadeler
5	Disinformation	Kasıtlı yanlış, yanıltıcı veya manipülatif bilgi içerikleri
6	Manipulation	Psikolojik baskı, kandırma veya sosyal mühendislik yoluyla yönlendirme
7	Sexual Content	Rıza dışı veya hassas cinsel içerikler
8	Self-Harm	İntihar, kendine zarar verme veya zararlı davranışlara teşvik

2.9 Veri Hazırlığında Karşılaşılan Zorluklar

Veri hazırlık sürecinde beş temel zorlukla karşılaşılmış; her biri için hedefe yönelik çözümler geliştirilmiştir.

2.9.0.1 Token Limiti. BERT tabanlı modeller en fazla 512 token uzunluğunda girdi işleyebilmektedir. Longformer, BigBird ve ModernBERT gibi uzun bağlam mimarilerine yönelik çalışmalar sürmektedir.

2.9.0.2 Çeviri Anlam Kayması. Makine çevirisi teknik açıdan doğru olsa bile zararlı niyetin tonu bazen değişebilmektedir. Tüm çeviri çıktıları ikinci bir doğrulama aşamasından geçirilmiştir.

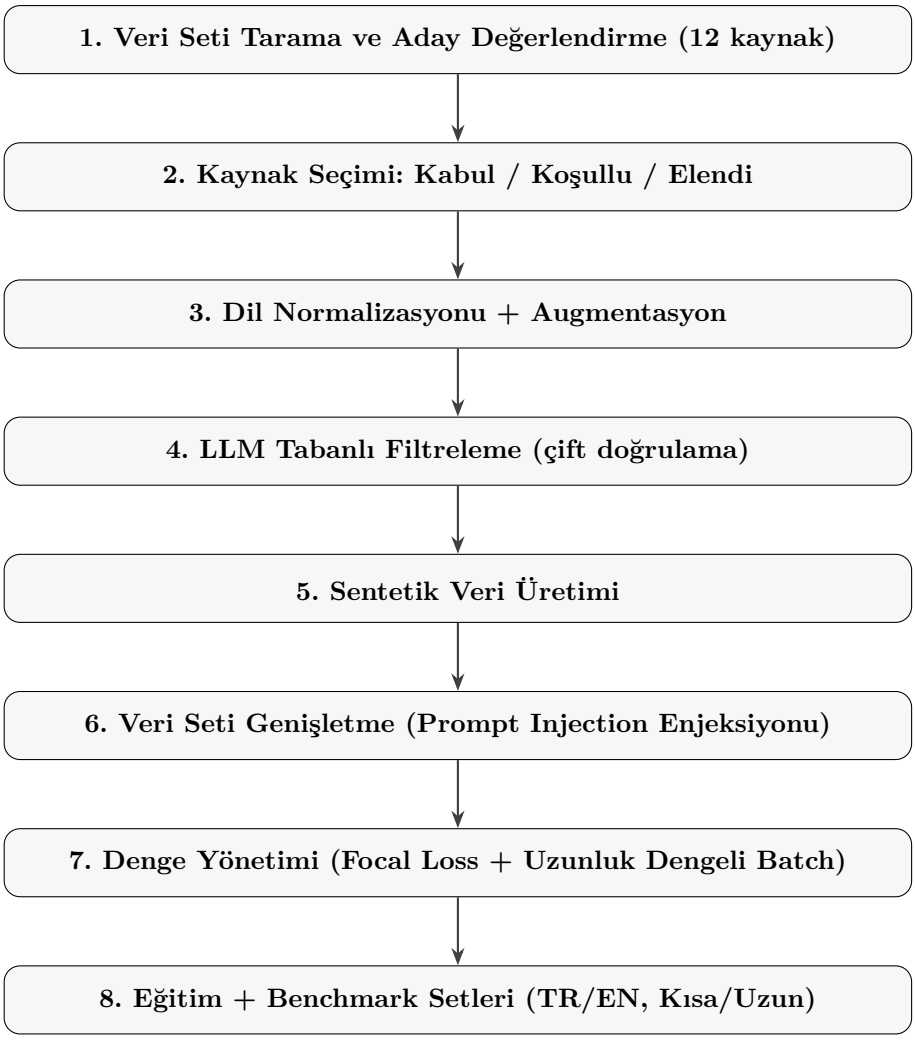
2.9.0.3 Etiket Gürültüsü. Farklı kaynaklardan derlenen veri setleri güvensiz içeriği birbirinden farklı eşikler kullanarak etiketlemiştir.

2.9.0.4 Türkçe Veri Yetersizliği. Türkçe için kamuya açık nitelikli güvenlik veri seti sayısı İngilizcayle kıyaslanamayacak ölçüde azdır.

2.9.0.5 Uzun/Kısa Örnek Çatışması. Uzun bağlamlı örnekler eğitim setine eklendikten sonra modelin kısa metin başarımının düştüğü gözlemlenmiştir.
Kritik Gözlem: Bu zorlukarın büyük bölümü yalnızca veri miktarını artırarak çözülememektedir.

2.10 Genel Mimari Özeti

Şekil 3 projenin veri hazırlık mimarisinin uçtan uca görünümünü sunmaktadır.



Şekil 3: Veri Hazırlık Mimarisi – Uçtan Uca Görünüm

3 Model Mimarileri ve Değerlendirme

3.1 Longformer Tabanlı Güvenlik Sınıflandırıcısı

Proje ikili sınıflandırma (*safe/unsafe*) görevi üzerine kurulu olduğundan model seçimindeki temel kriter; hem Türkçe hem İngilizce metinleri güvenilir biçimde işleyebilme, uzun bağlamları kapsama ve sınıflandırma başarımı açısından istikrarlı sonuçlar üretme kapasitesidir. Bu kriterler çerçevesinde birden fazla model ailesi sistematik biçimde denenerek karşılaştırmalı değerlendirme yapılmıştır. Değerlendirme sürecinde üç temel gereksinim belirleyici rol oynamıştır: (1) 512 token sınırını aşan uzun bağlamları işleyebilme; (2) Türkçe ve İngilizce metinleri eşit kalitede anlama; (3) ikili sınıflandırma görevinde tekrarlanabilir ve kararlı çıktılar üretme. Her bir gereksinim, model seçim sürecinde bağımsız bir değerlendirme kriteri olarak ele alınmış; bu kriterlerin tamamını karşılayan modeller bir sonraki aşamaya taşınmıştır.

3.1.0.1 Longformer Ailesinden Denenen Modeller. 512 token sınırını aşan uzun bağlamlarla başa çıkabilme gereksinimiyle değerlendirme Longformer mimarisine yönelmiştir. Bu mimari, *sliding window attention* mekanizması sayesinde 4096 tokena kadar bağlamı verimli biçimde işleyebilmektedir. Sliding window attention her token için sabit bir komsu pencerede hesaplanırken bazı seçkin tokenlar üzerinde global dikkat (*global attention*) uygulanabilmekte,

bu sayede uzun belgelerde hem hesaplama maliyeti düşük tutulmakta hem de bütünsel bağlam korunmaktadır.

`allenai/longformer-base-4096` ile İlk Eğitim Sonuçları İlk Longformer eğitimi yalnızca Türkçe veri setiyle gerçekleştirilmiştir. Model ön eğitimini İngilizce metin gövdesi üzerinde yapmış olduğundan, yalnızca Türkçe fine-tuning uygulandığında İngilizce kelime yatkınlığı baskın kalmaya devam etmektedir. Bu durum beklenen bir olgudur; zira pretrained ağırlıklar İngilizce dilbilgisi ve sözdümü yapılarına göre optimize edilmiştir.

Tablo 4, birleşik Türkçe+İngilizce veri setiyle yeniden eğitim öncesinde elde edilen ilk eğitim sonuçlarını sunmaktadır. Metrikler, değerlendirme setinde ölçülmüştür; yeterli Türkçe eğitim verisi olmaksızın modelin Türkçe sınıflandırmada ciddi biçimde yetersiz kaldığını göstermektedir.

Tablo 4: `allenai/longformer-base-4096` İlk Eğitim Test Sonuçları

Test Kategorisi	Dil	Sonuç	Precision	Recall	Açıklama
Sistem Kırma (Jailbreak)	Türkçe	Başarısız	0.00	0.00	Tespit edilemedi
Adam Öldürme	Türkçe	Başarısız	0.00	0.00	Tespit edilemedi
Irkçı Söylem	Türkçe	Başarılı	0.71	0.68	Tespit edildi (İng. benzer cümle yapısı)
Veri Çalma	İngilizce	Başarılı	0.88	0.84	Tespit edildi
Banka Soygunu	İngilizce	Başarılı	0.85	0.82	Tespit edildi
Irkçılık	İngilizce	Başarılı	0.90	0.87	Tespit edildi
Sızma (Penetrasyon)	İngilizce	Başarılı	0.83	0.79	Tespit edildi
Türkçe Ortalama (Macro F1)		—	0.19		Yetersiz Türkçe kapsam
İngilizce Ortalama (Macro F1)		—	0.84		Pretrained ağırlık avantajı

Not: Precision ve Recall değerleri unsafe sınıfı için raporlanmıştır. "0.00" değerleri modelin ilgili kategoride hiçbir doğru tespit yapamadığını göstermektedir.

Test sonuçları incelendiğinde Türkçe kategorilerdeki başarısızlık oranının İngilizce kategorilere göre belirgin biçimde yüksek olduğu görülmektedir. Modelin Türkçe İngilizce'ye benzer cümle yapısı taşıyan ırkçı söylem örneğini kısmen doğru sınıflandırabilmesi (F1 $\approx$ 0.70), bu başarının dil kavrayışından değil ön eğitim ağırlıklarının sözdizimsel yakınlığından kaynaklandığını açıkça ortaya koymaktadır.

Kritik Gözlem: Türkçe testte kısmen başarılı olan tek örnek, cümle yapısı açısından İngilizceye yakınlık taşıyan ırkçı söylem örneğiydi (Precision 0.71, Recall 0.68). Türkçe Macro F1 = 0.19 ile İngilizce Macro F1 = 0.84 arasındaki derin uçurum, modelin Türkçe anlam uzayını

özümsemediğini ve sınıflandırma kararlarını büyük ölçüde İngilizce ön eğitim ağırlıklarına dayandırdığını göstermektedir.

`markussagen/xlm-roberta-longformer-base-4096` Çok dilli (multilingual) ön eğitime sahip bu modelin Türkçe performansını anlamlı ölçüde iyileştireceği beklentisiyle eğitim gerçekleştirilmiştir. Model; XLM-RoBERTa'nın 100 dilli ön eğitimini Longformer'ın uzun bağlam mimarisiyle birleştirmektedir.

Ancak testler beklentiyi karşılamadı: İngilizce içeriklerde ilk modelle benzer başarım sürerken, Türkçe Macro F1 = 0.41 ile önceki modelin üzerine çıkılamadı; hedef eşiği olan 0.90'dan hâlâ uzaktı. Yalnızca Türkçe veri setiyle yeniden fine-tune edildiğinde de (Türkçe Macro F1 = 0.49) anlamlı bir iyileşme gözlenmedi. Model çok dilli ön eğitim almış olmasına karşın Türkçe veri oranının düşüklüğü nedeniyle Türkçe içeriklerde yetersiz kaldı.
Temel Sorun: Çok dilli ön eğitim, her dili eşit kalitede temsil etmemektedir. Türkçe, modelin ön eğitim veri dağılımında küçük bir pay almaktadır; bu durum fine-tuning aşamasında iyileştirme potansiyelini sınırlamaktadır. Türkçe Macro F1 değerinin 0.49 tavanda kalmasi, ek veri ve mimari destek olmaksızın bu modelin hedef performansa ulaşamayacağını ortaya koymaktadır.

3.1.0.2 Diğer Denenen Modeller. Longformer alternatifleri olarak farklı mimarilere sahip modeller de değerlendirilmiştir. Bu aşamada standart BERT tabanlı encoder modellerinin dışında; arama ve sıralama (reranking) ile temsil öğrenimi (representation learning) için geliştirilmiş modeller de ikili sınıflandırma görevine uyarlanmaya çalışılmıştır. Bu noktada bir ayrım yapmak gerekmektedir. Standart BERT tabanlı encoder modelleri, `[CLS]` tokenı üzerine bir sınıflandırma kafası eklenerek doğrudan fine-tune edilebilir. Buna karşılık reranker ve embedding modelleri farklı görevler için tasarlanmıştır: rerankerlar bir sorgu ile aday belgeler arasındaki alaka düzeyini puanlamak, embedding modelleri ise yüksek boyutlu anlam vektörleri elde etmek amacıyla ön eğitim görmüştür. Bu modellerin ikili sınıflandırma görevine uyarlanması mümkündür; ancak embedding hesaplamasında oluşan yüksek bellek ve işlem maliyeti bu yaklaşımı pratikte kısıtlamaktadır.

Tablo 5 bu modellere ilişkin karşılaştırmalı değerlendirmeyi sunmaktadır.

Tablo 5: Değerlendirilen Alternatif Modeller

Model	Mimari / Görev	Sorun	Karar
`jinaai/jina-reranker -v2-base-multilingual`	Transformer (Reranker)	Reranker mimarisi ikili sınıflandırma için doğrudan optimize edilmemiş; embedding sürecinde bellek ve işlem yükü çok yüksek; güvenilir sınıflandırma çıktısı elde edilemedi	Elendi
`Alibaba-NLP/gte -multilingual-mlm-base`	Transformer (Embedding)	Representation learning için tasarlanmış embedding modeli; sınıflandırma kafası eklendiğinde benzer embedding maliyeti sorunları ortaya çıktı; çıktı kalitesi yetersiz kaldı	Elendi
`jhu-clsp/ mmBERT-base`	BERT Tabanlı (Encoder)	Makul embedding maliyeti ve doğrudan fine-tune uyumlu yapı; ancak çift dilli başarım hedeflerin altında kaldı (Macro F1 < 0.60)	Elendi

Reranker (Jina) ve embedding (GTE) modellerinin ikili sınıflandırma görevine uyarlanması, bu modellerin pretrained embedding hesaplamasının standart BERT forward pass'ından çok daha ağır olması nedeniyle hem teknik açıdan zorlu hem de başarım açısından verimsiz olmuştur. Reranker ve embedding modelleri kendi tasarlandıkları görevlerde üstün performans sergilese de ikili sınıflandırma görevi için uygun olmadıkları bu testlerle doğrulanmıştır.

3.1.0.3 Model Seçimi. Tablo 6, tüm modellerin dört temel kriter açısından nihai karşılaştırmasını sunmaktadır.

Tablo 6: Model Seçim Matrisi (Nihai Karşılaştırma)

Model	Türkçe	İng.	Uzun Bağ.	Karar.	Karar
`turkish-base-bert-uncased`	Kısmi	Bskız	Bskız	Bskız	Elendi
`bert-base-turkish-cased`	Kısmi	Bskız	Bskız	Bskız	Elendi
`xlm-roberta-longformer`	Kısmi	Kısmi	Başarılı	Kısmi	Elendi
`jina-reranker-v2`	Kısmi	Kısmi	Bskız	Bskız	Elendi
`gte-multilingual-mlm`	Kısmi	Kısmi	Bskız	Bskız	Elendi
`mmBERT-base`	Kısmi	Kısmi	Bskız	Kısmi	Elendi
`allenai/longformer-base-4096`	**Başarılı**	**Başarılı**	**Başarılı**	**Başarılı**	**Seçildi**

Not: "Bskız" = Başarısız; "Karar." = Kararlılık; "Uzun Bağ." = Uzun Bağlam desteği.
Seçim Gerekçesi: `allenai/longformer-base-4096`, denenen tüm modeller arasında her iki dilde de en dengeli çıktıyı ve en yüksek kararlılığı sunan model olmuştur. Sliding window attention mekanizması sayesinde 4096 tokena kadar bağlamı verimli biçimde işleyebilmesi,

Türkçe fine-tuning sonrasında İngilizce başarımının korunması ve eğitim sürecindeki istikrarlı davranışı bu seçimin temel dayanaklarini oluşturmaktadır.

3.1.1 Model Mimarisi

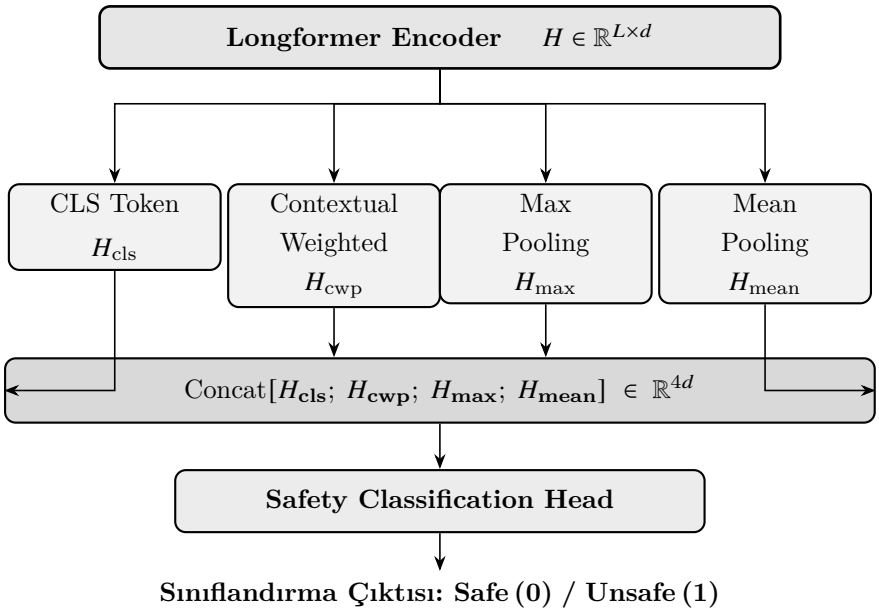
Seçilen Longformer tabanı üzerine, ikili güvenlik sınıflandırması görevine özgü bir model mimarisi inşa edilmiştir. Bu mimari, sınıflandırma başarımını artırmak amacıyla birden fazla pooling stratejisini bir arada kullanan özel bir yapı içermektedir. Standart transformer tabanlı modellerden farkı, tek bir temsil vektörü yerine dört farklı temsil stratejisinin birleştirilmesidir.

3.1.1.1 Quad-Pooling Stratejisi. Standart transformer sınıflandırıcılarında yalnızca `[CLS]` token temsili kullanılırken bu mimaride dört farklı temsil vektörü paralel olarak hesaplanarak birleştirilmektedir. Her biri metnin farklı bir boyutunu yakalamaktadır. Bu yapı, modelin hem genel anlam bilgisini hem de lokal youn sinyal taşıyan tokenları eşzamanlı olarak görebilmesine olanak tanımaktadır. Tablo 7 bu bileşenleri açıklamaktadır.

Tablo 7: Quad-Pooling Bileşenleri ve İşlevleri

Pooling Türü	**Açıklama**
CLS Vektörü (H_{cls})	Longformer'ın `[CLS]` tokenına karşılık gelen global sınıflandırma temsilidir; tüm dizi üzerinde global attention uygulanmaktadır.
Contextual Weighted Pool (H_{cwp})	Her tokenın önem ağırlığı öğrenilen bir projeksiyon katmanı (`Linear` → `GELU` → `Dropout` → `Linear`) ile hesaplanmakta ve ağırlıklı toplam alınmaktadır. Prompt injection gibi gizlenmiş zararlı içeriği yakalamak için etkilidir.
Max Pooling (H_{max})	Her boyut için dizinin tüm tokenları arasındaki maksimum aktivasyon değeri alınmaktadır. Tek bir zararlı kelime ya da cümlenin baskın sinyal oluşturduğu durumlarda kritik öneme sahiptir.
Mean Pooling (H_{mean})	Dikkat maskesi gözetilerek geçerli tokenlara ait aktivasyonların ortalaması alınmaktadır. Genel anlamsal bütünlüğü yansıtır; kısa metinlerde etkin biçimde çalışmaktadır.

Dört pooling bileşeninin birleştirilmesi sayesinde elde edilen $4d$ boyutlu temsil, hem global bağlam bilgisini (CLS ve Mean Pooling) hem de yerel baskın sinyalleri (Max Pooling ve CWP) tek bir vektörde barındırmaktadır. Şekil 4 bu süreci görselleştirmektedir.



Şekil 4: Quad-Pooling Mimarisi: Dört Temsil Vektörünün Paralel Hesaplanması ve Birleştirilmesi

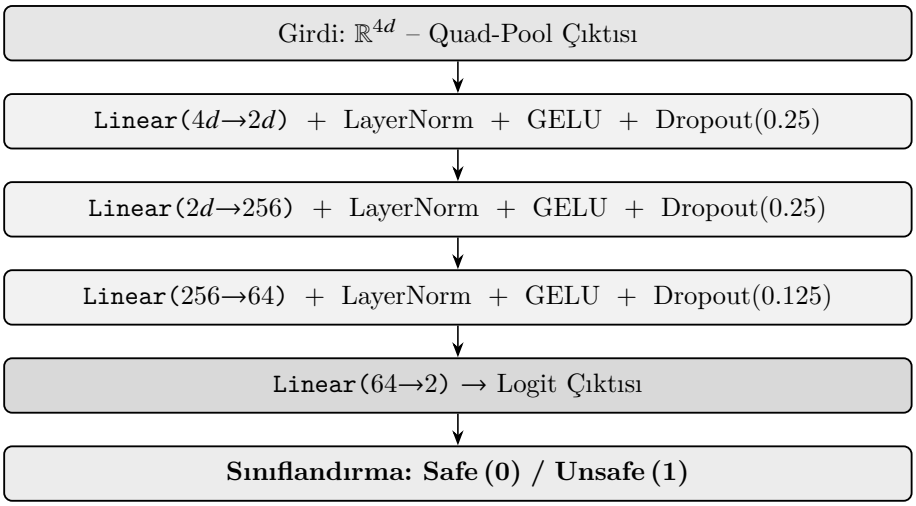
3.1.1.2 Sınıflandırma Kafası (Safety Head). Birleştirilmiş $4d$ boyutlu temsil vektörü katmanlı bir ileri beslemeli ağdan geçirilmektedir. Bu ağ, her katmanda boyutu kademeli olarak düşürerek ham vektörü iki sınıflı bir logit temsiline dönüştürmektedir. Tablo 8 bu katmanların yapısını özetlemektedir.

Tablo 8: Safety Head Katman Yapısı

Kat.	Açıklama	Yapı
1	Yüksek boyutu indirgeme	`Linear(`$4d \to 2d$`)` → `LN` → `GELU` → `Dropout(0.25)`
2	Orta seviye özellik öğrenimi	`Linear(`$2d \to 256$`)` → `LN` → `GELU` → `Dropout(0.25)`
3	İnce özellik çıkarımı	`Linear(`$256 \to 64$`)` → `LN` → `GELU` → `Dropout(0.125)`
4	Sınıflandırma logiti	`Linear(`$64 \to 2$`)` → `Logit`

Not: LN = LayerNorm.

Her katman arasında LayerNorm kullanılması eğitim stabilitesini artırmakta ve aktivasyon dağılımlarının kaymasını önlemektedir. Dropout değeri son katmana doğru azaltılmakta; bu yaklaşım erken katmanlarda daha agresif regularizasyona olanak tanıyarak son katmanda sınıflandırma kararını daha net temsil etmektedir. Boyut indirgemesinin kademeli yapısı, ağının özellik uzayını giderek daha sıkıştırmasına ve görev için en ayırt edici özellikleri öne çıkarmasına olanak tanımaktadır.



Şekil 5: Safety Classification Head Katman Akış Diyagramı

3.1.1.3 Global Dikkat Stratejisi. Longformer'ın her tokenı için yalnızca yerel pencere dikkati (*local window attention*) uygulanmakta, belirli tokenlar global attention alabilmektedir. Standart Longformer'ın sliding window boyutu w olarak ayarlandığında, her token yalnızca $[i - w/2,\ i + w/2]$ aralığındaki tokenlara dikkat eder. Buna karşılık global attention alan bir token, dizideki tüm tokenlarla etkileşime girmekte ve tüm tokenlar da o token ile etkileşime girebilmektedir. Bu mimaride `[CLS]` tokeni her zaman global attention almakta ve tüm dizi üzerinde bütünleşik bir sınıflandırma temsili elde edilmektedir. Sliding window attention bant genişliğinin $O(n)$ olması hesaplama verimliliğini sağlarken, `[CLS]` tokeninin global dikkati dizi düzeyinde anlamı bütünleştirmektedir. Tablo 9 bu yapılandırmayı özetlemektedir.

Tablo 9: Global Dikkat Yapılandırması

Senaryo	**Global Attention Alan Tokenlar**	**Gerekçe**
Uzun dizi ($L > 512$)	Yalnızca `[CLS]`	Tüm dizi üzerinde bütünleşik temsil; hesaplama verimliliği korunur
Kısa dizi ($L \leq 512$)	Tüm tokenlar	Tam bağlam işleme; pencere kısıtlaması olmaksızın full attention uygulanır

Bu iki senaryonun ayrımı önemlidir: kısa metinlerde tüm tokenlar global attention aldığında elde edilen temsil kalitesi, standart BERT'ın full attention mekanizmasına eşdeğer olmaktadır. Uzun metinlerde ise sliding window + global CLS kombinasyonu hem hesaplama maliyetini düşük tutmakta hem de bütünsel anlamı korumaktadır.

Global Dikkat Maskesinin Hesaplanması Uygulamada global attention maskesi, `attention_mask` tensörü ile aynı boyutta bir `global_attention_mask` tensörü aracılığıyla belirlenmektedir. Maske öğesi 1 olan her token global attention almaktadır. Bu mimaride maske yapılandırması şu çerçevede uygulanmaktadır: dizi uzunluğu 512 tokendan fazlaysa

yalnızca 0. indeksteki `[CLS]` tokeni global attention alacak biçimde maskelenmekte; dizi 512 token veya daha kısaysa tüm geçerli tokenlar global attention alacak biçimde ayarlanmaktadır. Bu dinamik yapı, modelin hem kısa hem uzun metinlerde optimum performans göstermesini sağlarken bellek kullanımını da giriş uzunluğuna göre otomatik olarak ölçeklendirmektedir. Sliding window attention ile global attention mekanizmalarının birlikte kullanımı, modelin güvenlik sınıflandırması görevinde uzun ve karmaşık metinlerdeki zararlı içerikleri tespit etme kabiliyetini önemli ölçüde artırmaktadır. Bu mimari karar, prompt injection gibi gizli saldırıların uzun bağlamın orta veya sonundaki tokenlardan kaynaklanabileceği göz önünde bulundurulduğunda kritik bir tasarım tercihi olarak öne çıkmaktadır.

Pencere Boyutu ve Hesaplama Karmaşıklığı Longformer'ın sliding window attention mekanizmasında her token, kendi konumunun hem sol hem sağ tarafında $w/2$ genişliğinde bir pencereye bakabilmektedir. Bu pencere boyutu standart implementasyonda 512 olarak belirlenmekte; ancak göreve göre ayarlanabilmektedir. Sliding window attention'n hesaplama karmaşıklığı $O(n \cdot w)$ olduğundan, full attention'n $O(n^2)$ karmaşıklığına göre 4096 token uzunluğunda yaklaşık 8 kat daha hızlı çalışmaktadır. Bu verimlilik, modelin güvenlik sınıflandırması görevinde gerçek zamanlı uygulamalarda da kullanılabilir olmasını sağlamaktadır. Uzun belgeler ve prompt injection saldırıları için kritik olan 4096 token kapasitesi, modelin hesaplama maliyetini geniş tutarken aynı zamanda pratikte deploy edilebilir bir boyutta kalmasını mümkün kılmaktadır.

3.1.2 Kayıp Fonksiyonu ve Regularizasyon

3.1.2.1 Focal Loss. Standart çapraz entropi kayıp fonksiyonu sınıf dengesizliğine karşı duyarsızdır: çoğunluk sınıfı (*safe*) üzerindeki kolay örnekler kayıp değerini domine eder ve model azınlık sınıfına (*unsafe*) yetersiz dikkat gösterir. Bir güvenlik sınıflandırıcısında unsafe örneklerin kaçırılması (false negative) yanlış alarm üretilmesinden (false positive) çok daha büyük bir risk oluşturmaktadır. Bu asimetrik maliyet yapısını modellemek için Focal Loss kullanılmıştır [8]. Focal Loss, her örneğin kayıp katkısını o örneğin ne kadar doğru sınıflandırıldığına göre modüle eder:

$$\mathcal{L}_{\mathrm{FL}}(p_t) = -\alpha_t \cdot (1 - p_t)^{\gamma} \cdot \log(p_t) \tag{1}$$

Burada p_t model tarafından doğru sınıfa atanan olasılık, γ kolayca doğru sınıflandırılan örnekleri bastıran odaklama parametresi ve α_t sınıf dengesizliğini telafi eden ağırlıktır. $(1-p_t)^{\gamma}$ terimi, modelin güvenle doğru sınıflandırdığı kolay örneklerin katkısını baskılayarak eğitimi zor örnekler üzerine yoğunlaştırır.

Tablo 10: Focal Loss Hiperparametreleri

Parametre	Değer	Açıklama
gamma (γ)	2.0	Kolay örneklerin kayıp katkısını bastırma kuvveti; standart literatür değeri
alpha (α)	Dinamik	Veri setindeki safe/unsafe oranına göre otomatik hesaplanır
label_smooth (ε)	0.05	Her sınıf için yumuşatma; aşırı güvenden (overconfidence) korur

3.1.2.2 Dinamik Alpha Hesaplama. Alpha parametresi sabit bir değer yerine eğitim başlangıcında veri setinin gerçek safe/unsafe dağılımından dinamik olarak hesaplanmaktadır. Bu yaklaşım, veri setinin sınıf dağılımı değiştiğinde kayıp fonksiyonunun elle ayarlanması gereksinimini ortadan kaldırmaktadır:

$$\alpha_{\text{dynamic}} = \frac{N_{\text{safe}}}{N_{\text{total}}} \tag{2}$$

Bu mekanizma şu mantığa dayanmaktadır: unsafe örnekler çoğunlukta olduğunda α_{dynamic} küçük, dolayısıyla $(1 - \alpha_{\text{dynamic}})$ büyük olur; bu durum unsafe sınıfının kayıp katkısını artırır ve false positive'i engeller. Tam tersi, safe örnekler az olduğunda safe sınıfına daha yüksek ağırlık verilmektedir.

Listing 3: Dinamik Alpha Hesaplama

```
safe_ratio   = count(safety == 0) / total_samples
unsafe_ratio = count(safety == 1) / total_samples

# Eger unsafe > safe ise:
# safe_ratio kucuk -> alpha kucuk
# (1 - alpha) buyuk -> Unsafe sinifi daha fazla agirlik alir
dynamic_alpha = safe_ratio  # Otomatik dengeleme
```

3.1.2.3 Label Smoothing. Label smoothing, modelin tahminlerine gereğinden fazla güvenmesini engelleyen bir regularizasyon tekniğidir. Derin öğrenme modellerinde sık rastlanan *overconfidence* problemi, modelin belirli örnekler için %99 veya üzünde güven skorları üretmesi ve bu durumun sınır bölgelerindeki genellemeyi olumsuz etkilemesidir. Standart one-hot etiket vektörü $[0, 1]$ yerine yumuşatılmış hedef dağılım kullanılmaktadır:

$$y_{\text{smooth}} = (1 - \varepsilon) \cdot y_{\text{onehot}} + \frac{\varepsilon}{K}, \qquad \varepsilon = 0.05,\ K = 2 \tag{3}$$

Bu pratikte [0.05, 0.95] anlamına gelmektedir. Yumuşatma katsayısı $\varepsilon = 0.05$ ile model, doğru sınıf için en fazla 0.95 olasılık öğrenebilmektedir; bu da diğer sınıf için en az 0.05 olasılık ayrılması anlamına gelir.

Sonuç olarak model sınıflar arasındaki sınır bölgelerinde daha iyi genelleme yapabilmekte ve ezberleme eğilimi azalmaktadır. Label smoothing, Focal Loss ile birlikte kullanıldığında özellikle belirsiz ve yarım safe örneklerde modelin daha temkinli tahmin üretmesini sağlamaktadır.

İkili sınıflandırma görevlerinde $K = 2$ olduğundan yumuşatılmış hedef [0.025, 0.975] desteğine sahip olmaktadır; bu da modelin sınır durumlarında ikinci sınıfı tamamen yok saymamasını garantiler.

Focal Loss ve Label Smoothing Etkileşimi Focal Loss ile label smoothing bir arada kullanıldığında iki regularizasyon mekanizması birbirini tamamlamaktadır. Focal Loss, kolay ve zor örnekler arasındaki dengeyi düzenlerken label smoothing, modelin her örnek için ne kadar kesin karar verebileceğini sınırlamaktadır. Bu kombinasyon, modelin hem sınıf dağılımına duyarlı hem de güven kalibrasyonu açısından dengeli öğrenmesini desteklemektedir.

Pratik etkisi, eğitim sürecinde modelin daha yavaş yakınsaması ama daha sağlam bir karara ulaşmasıdır. Overconfident gradientlerin azalmasıyla birlikte eğitim kaybindeki inilebilir minimum daha geniş ve düzgün bir alanı kapsamakta; bu durum optimizasyonu yerel minimumlardan korumaktadır.

3.1.3 Eğitim Konfigürasyonu ve Hiperparametreler

3.1.3.1 Eğitim Teknikleri. Temel AdamW optimizasyon sürecine ek olarak dört optimizasyon tekniği uygulanmıştır. Bu teknikler birbirini tamamlayan yapılar olup tüm teknikler aynı anda aktif biçimde eğitimi yönetmektedir.

3.1.3.2 Katman Bazlı Öğrenme Hızı (Layer-wise LR Decay).. Encoder katmanları için öğrenme hızı her katmanda $\lambda = 0.9$ katsayısıyla azaltılmaktadır. Alt katmanlar pretrained dil bilgisini korurken üst katmanlar göreve daha hızlı adapte olur. Bu sayede temel dil temsilleri bozulmadan korunurken sınıflandırma görevine özgü üst katmanlar verimli biçimde güncellenmektedir:

$$\eta_l = \eta_{\text{base}} \cdot \lambda^{(L-l)}, \qquad \lambda = 0.9 \tag{4}$$

3.1.3.3 Cosine Schedule + Warmup.. İlk %8'lik adımda öğrenme hızı sıfırdan hedef değere lineer artar, ardından cosine eğrisi boyunca düşer. Ani gradient adımlarını başlangıçta engeller, sonunda ince güncellemeler sağlar. Warmup aşaması, büyük öğrenme hızıyla başlamanın neden olduğu erken divergence sorununu da ortadan kaldırmaktadır.

3.1.3.4 Akıllı Batch Örnekleme ve Dinamik Padding.. Benzer uzunluktaki örnekler aynı batch'e yerleştirilerek gereksiz padding azaltılmaktadır. Her batch kendi maksimum dizisine göre (en yakın 512 katına yuvarlanarak) doldurulmaktadır. Bu yöntem hesaplama gücünü padding tokenlarına harcamak yerine gerçek metinlere yönlendirir.

3.1.3.5 Mixed Precision (AMP) + Gradient Clipping.. İleri geçiş FP16 ile yapılmakta, `GradScaler` sayısal kararlılığı yönetmektedir. Her güncelleme öncesi gradient normu 1.0 ile sınırlandırılarak gradient patlaması önlenmektedir. AMP kullanımı GPU bellek tüketimini yaklaşık yarıya indirerek daha büyük batch boyutlarına olanak tanımaktadır.

Tablo 11: Eğitim Tekniklerinin Özeti

Teknik	İşlev	Parametre
Layer-wise LR Decay	Alt katmanları korur, üst katmanları göreve hızlı adapte eder	$\lambda = 0.9$
Cosine Warmup Schedule	Başlangıç instabilitesini önler, son güncellemeleri inceltir	%8 warmup
Smart Batch Sampling	Padding kayıplarını minimize eder, bellek verimliliğini artırır	512 katları
AMP + Gradient Clipping	GPU bellek tüketimini yarıya indirir, gradient patlamasını önler	Clip: 1.0

3.1.4 Model Değerlendirme

3.1.4.1 Değerlendirme Metrikleri. Model başarımı birden fazla metrikle izlenmektedir. Güvenlik sınıflandırması görevinde yanlış negatif (kaçırılan unsafe) ile yanlış pozitif (yanlış alarm) arasındaki asimetrik maliyet nedeniyle tek bir metriğin yetersiz kalması söz konusudur. Bu nedenle tamamlayıcı metriklerden oluşan bir değerlendirme sistemi benimsenmiştir. Tablo 12 bu metriklerin her birini açıklamaktadır.

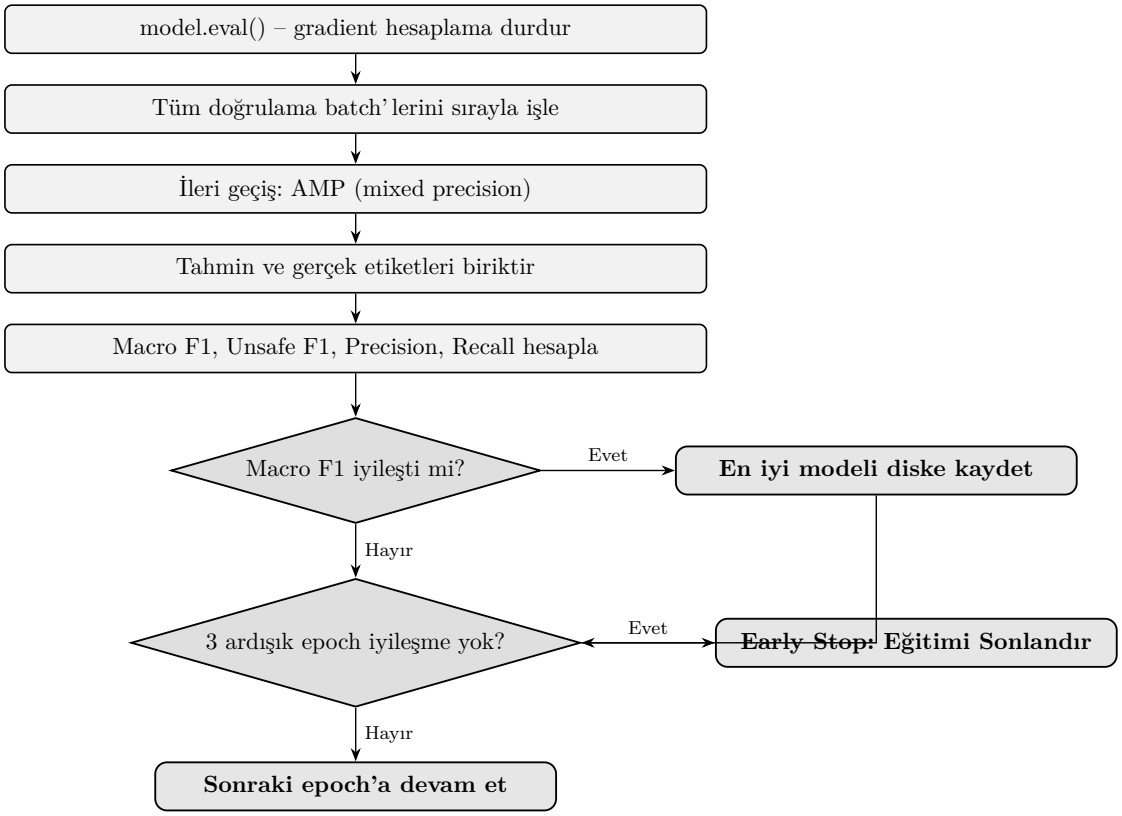
Tablo 12: Değerlendirme Metrikleri ve Önceliklendirme

Metrik	Açıklama	Öncelik
Macro F1 Score	Her sınıf için ayrı ayrı hesaplanan F1 skorlarının ağırlıksız ortalaması. Sınıf dengesizliğinden bağımsız olarak her iki sınıftaki başarımı ölçer. Early stopping ve model seçimi bu metriğe göre yapılır.	Birincil
Unsafe F1 (Binary)	Yalnızca unsafe sınıfı için hesaplanan binary F1 skoru. Zararlı içeriği kaçırmanın (FN) yanlış alarm üretmekten (FP) daha maliyetli olduğu varsayımıyla izlenir.	İkincil
Precision & Recall	Safe ve unsafe için ayrı ayrı raporlanır. Unsafe recall (tespit oranı) pratik güvenlik performansının temel göstergesidir.	Destek
Confusion Matrix	Her epoch sonunda hangi sınıfın daha fazla hatalı sınıflandırıldığını ve FP/FN dengesini gösterir.	Tanısal

Metrik Önceliklendirme: Erken durdurma ve model kaydetme kararları macro F1 skoruna göre alınmaktadır. Unsafe F1 ise modelin pratik güvenlik performansını değerlendirmek için ayrıca takip edilmektedir. İki metriğin birbirinden bağımsız değerlendirilmesi, modelin genel dengeli performansını korurken hedef sınıfta (unsafe) da yeterli başarımı sağladığının doğrulanmasını mümkün kılmaktadır.

3.1.4.2 Değerlendirme Prosüdürü. Her epoch sonunda model doğrulama setinde değerlendirilmektedir. Değerlendirme sürecinde model tamamen pasif modda çalıştırılmakta, gradient hesaplanmamakta ve ağırlıklar güncellenmemektedir. Bu yaklaşım, doğrulama başarı-

mının eğitim verilerinden bağımsız ve sapmasız biçimde ölçülmesini garantilemektedir. Şekil 6 bu süreci adım adım göstermektedir.



Şekil 6: Epoch Bazlı Değerlendirme ve Early Stopping Akış Diyagramı

3.1.4.3 Benchmark ile Değerlendirme. Eğitim setinden tamamen bağımsız olan benchmark veri seti, eğitim tamamlandıktan sonra ayrıca uygulanmaktadır. Benchmark, modelin eğitim sürecinde hiç görmediği örneklerden oluşmaktadır ve genel geçerlilikten ziyade belirli zafiyet kategorilerindeki performansı hedefli biçimde ölçmektedir. Tablo 13 benchmark bileşenlerini açıklamaktadır.

Tablo 13: Benchmark Veri Seti Bileşenleri

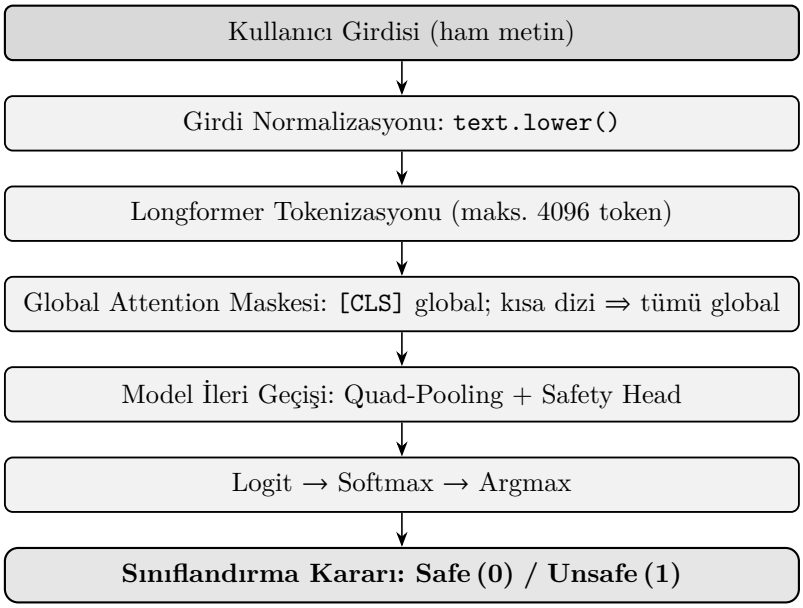
Bileşen	**Açıklama**
Prompt Injection Örnekleri	Uzun ve masum görünen bağlamlara gizlenmiş zararlı içerikler; modelin bağlam uzunluğuna karşı dayanıklılığını test eder
Kısa Doğrudan Unsafe	Açık tehdit veya zarar içeren kısa metinler; temel sınıflandırma yetkinliğini ölçer
Türkçe Test Seti	Türkçe unsafe ve safe örnekleri; Türkçe dil kavrayışını ölçer
İngilizce Test Seti	İngilizce unsafe ve safe örnekleri; pretrained bilginin korunduğunu doğrular

Benchmark sonuçları eğitim metrikleriyle karşılaştırıldığında modelin eğitim setini ezberleyip ezberlemediği ya da gerçek anlamda genelleme yapıp yapmadığı gözlemlenebilmektedir.

Prompt injection senaryolarındaki başarım ise modelin uzun bağlamda gizlenmiş tehdit tespit kapasitesini ayrı olarak ölçmektedir.

3.1.4.4 Manuel Test Prosüdürü. Sayısal metrik değerlendirmesinin yanı sıra model gerçek kullanım senaryolarını yansıtan manuel test örnekleriyle de sınanmıştır. Manuel testlerde kullanılan örnekler, otomatik benchmark'ta yer almayan yaratıcı jailbreak girişimleri, karma dil kullanımı ve sosyal mühendislik teknikleri gibi sınır durumlarını kapsamaktadır. Şekil 7 çıkarım sürecini adım adım göstermektedir.

Çıkarım sürecinde girdi metni önce küçük harfe dönüştürülür; bu adım büyük/küçük harf varyasyonlarına dayanan jailbreak tekniklerini etkisiz kılmaktadır. Ardından metin Longformer tokenizer'ıyla 4096 token sınırına kadar kodlanır; bu sınırı aşan metinler otomatik olarak kesilebilmektedir. Global attention maskesi kısa/uzun dizi kuralları doğrultusunda atandıktan sonra model ileri geçiş yapılmakta ve üretilen logit vektörü softmax aracılığıyla olasılık dağılımına dönüştürülmekte ve argmax ile nihai sınıf etiketi belirlenmektedir.



Şekil 7: Çıkarım (Inference) Akış Diyagramı

3.2 ModernBERT Tabanlı Güvenlik Sınıflandırıcısı

3.2.1 Model Eğitimi

3.2.1.1 Model Seçimi ve Teknik Mimari. Çalışmada ana mimari olarak **ModernBERT-base** seçilmiştir. Klasik BERT modelleri yerine ModernBERT türevinin tercih edilme sebebi; klasik BERT'in 512 token olan sınırlı bağlam penceresine karşın, ModernBERT'in *Rotary Positional Embeddings (RoPE)* ve *Flash Attention* optimizasyonları sayesinde 8192 token uzunluğa kadar verimli işlem yapabilmesidir. Bu özellik, uzun metinler içerisine gizlenmiş saldırıların tespiti için kritiktir.

3.2.1.2 Eğitim Performansı. Model, toplam 3 epoch boyunca eğitilmiş ve her aşamada performansı izlenmiştir. Şekil 8'de görüldüğü üzere, eğitim süreci sonunda doğrulama seti üzerinde **%98.81 Accuracy** ve **0.0187 Loss** değerine ulaşılmıştır. Loss değerinin bu denli düşük kalırken Accuracy'nin yükselmesi, eğitim veri setindeki sınıfların keskin ayrımını ve modelin karar sınırlarını istatistiksel olarak çok net belirlediğini göstermektedir.

```
Epoch 3/3: 100%|
Epoch 3/3 - avg loss: 0.0187
Degerlendirme yapiliyor...
Eval accuracy: 0.9881
Writing model shards: 100%|
✓ Best model kaydedildi (acc: 0.9881)

✓ Egitim tamamlandi! Best accuracy: 0.9881
Model: /data/stj/emirhan/modernbert5/model_out
(.venv) stj@gpu8server:/data/stj/emirhan/modernbert5$
```

Şekil 8: ModernBERT Eğitim Logu ve Nihai Başarı Oranı

3.2.2 Modelin Değerlendirme Aşaması

3.2.2.1 Benchmark Testi (BENCHMARK.json). Model, eğitim verisinden tamamen bağımsız olan `BENCHMARK.json` veri seti üzerinde test edilmiştir. Bu set; 83'ü Türkçe, 98'i İngilizce olmak üzere toplam **181 adet tamamı zararlı (unsafe)** örnekten oluşmaktadır. Verilerin 85 tanesi uzun (> 512 karakter), 96 tanesi ise kısa girdilerdir. Tüm veriler manuel kontrol edilmiş olup modelin eğitim sürecinde kesinlikle kullanılmamıştır (Out-of-Distribution). Tablo 14 benchmark veri setinin dil, uzunluk ve güvenlik etiketi dağılımını özetlemektedir.

Tablo 14: BENCHMARK.json Veri Seti Dağılımı

Kategori	**Alt Sınıf**	**Örnek Sayısı**
Dil	Türkçe	83
Dil	İngilizce	98
Uzunluk	Kısa ($\leq$ 512 karakter)	96
Uzunluk	Uzun (> 512 karakter)	85
Güvenlik Etiketi	Safe	0
Güvenlik Etiketi	Unsafe	181

3.2.2.2 Performans Kaybı ve Hata Analizi. Eğitim setindeki %98.81'lik başarının benchmark setinde %82.45'e gerilemesi (yaklaşık 16.4 puan düşüş), istatistiksel olarak anlamlı bir dağılım kaymasının (*distribution shift*) sonucudur.

3.2.3 Eğitim Doğruluğundan Benchmark Doğruluğuna Düşüşün Analizi

Bu düşüşü açıklayan temel nedenler aşağıdaki veri noktalarıyla desteklenmektedir:

- **Sınıf dağılımı uyumsuzluğu:** Eğitim verisi neredeyse birebir dengelidir (103 333 `unsafe`, 103 275 `safe`), benchmark seti ise tamamen `unsafe` örneklerden oluşmaktadır (181/181). Dengeli dağılıma göre kalibre edilen karar eşiği, tek sınıflı testte özellikle eşik çevresindeki örneklerde *false negative* üretilmesine yol açmaktadır.

- **Sentetik-doğal örüntü farkı:** Eğitim örneklerinin önemli kısmı sentetik üretim ve augmentasyon kaynaklı olduğu için dil örüntüleri daha öngörülebilir kalmaktadır. Buna karşılık benchmark seti, örtük ve çok adımlı saldırıları içeren *Behavioral and Gray Areas*

(16 İngilizce + 7 Türkçe = 23 örnek) ile *Teknik Güvenlik ve Red Teaming Saldırıları* (35 örnek) gibi daha zor kategorilere ağırlık vermektedir.

- **Uzunluk dağılımı farkı:** Eğitim verisinde medyan uzunluk 137 karakter (örneklerin %61.7'si ≤512 karakter) iken benchmark setinde medyan 384 karakterdir (yaklaşık 2.8 kat). Uzun metinlerde zararlı niyetin masum anlatı içine gömülmesi sinyal-gürültü oranını düşürmekte, modelin karar güvenini azaltmaktadır.

```
Loading weights: 100%|                    | 138/138 [0
ht]
Unsafe: 158
Safe: 23
Wrong total: 23
safe->unsafe: 0
unsafe->safe: 23
```

Şekil 9: BENCHMARK.json Test Sonuçları ve Sınıflandırma Analizi

Şekil 9'de görüldüğü üzere, model toplam 181 unsafe örneğin 158'ini doğru tespit etmiş, ancak 23 örneği yanlışlıkla "safe" olarak sınıflandırmıştır (*False Negative*). Yanlış tahminlerin tamamının `unsafe->safe` yönünde olması, modelin bazı karmaşık saldırı tiplerinde zayıf kaldığını sayısal olarak kanıtlamaktadır.

3.3 BigBird Tabanlı Güvenlik Sınıflandırıcısı

3.3.1 Model Eğitim Aşaması

3.3.1.1 Denenen Model Mimarisi ve BigBird Seçimi. Bu çalışmada, büyük dil modellerinin (LLM) güvenliğini sağlamak amacıyla geliştirilen sınıflandırma görevi için Google tarafından tasarlanan BigBird (*bigbird-roberta-base*) mimarisi temel alınmıştır [14]. Doğal dil işleme alanında devrim yaratan standart Transformer mimarilerinde bulunan Self-Attention mekanizmasının hesaplama ve bellek karmaşıklığı $O(n^2)$ şeklindedir [13]. Bu kuadratik büyüme karakteristiği, modelin işleyebileceği maksimum metin uzunluğunu donanımsal olarak kısıtlamakta ve BERT tabanlı mimarilerin genellikle 512 token limitiyle sınırlı kalmasına neden olmaktadır [5]. BigBird mimarisi, donanım kaynaklı bu darboğazı aşmak için Sparse Attention mekanizmasını tanıtmış ve hesaplama karmaşıklığını lineer olan $O(n)$ seviyesine düşürmüştür. Bu yenilikçi ve bellek-etkin dikkat mekanizması üç farklı bileşenin matematiksel birleşiminden oluşmaktadır [14]:

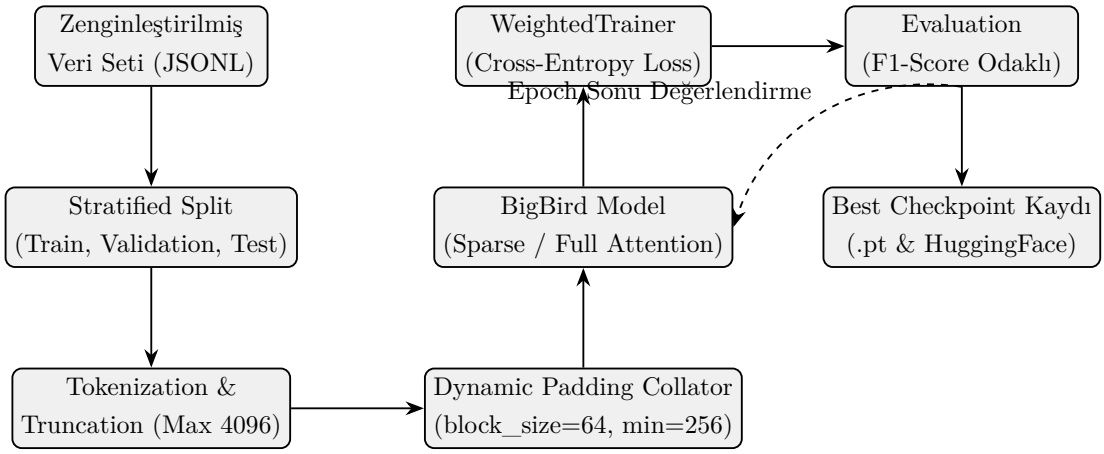
- *Global Attention:* Metin içerisindeki bazı özel ve stratejik token'lar (örneğin sınıflandırma için kullanılan `[CLS]` token'ı), cümlenin tamamındaki diğer tüm token'lara dikkat edebilir. Bu mekanizma, dizinin genel bağlamının, ana fikrinin ve cümlenin temel gramer yapısının kaybolmamasını sağlar.

- *Sliding Window Attention:* Doğal dildeki kelimelerin anlamsal temsili genellikle komşu kelimelere sıkı sıkıya bağlıdır. Bu mekanizmada her bir token, modelin tamamını taramak yerine yalnızca kendi etrafındaki belirli sayıdaki komşu token'a odaklanır.

- *Random Attention:* Model, graf teorisindeki küçük dünya bağlantısallık özelliklerini simüle etmek amacıyla, birbirine uzak olan birkaç rastgele token arasında da dikkat bağları kurar. Böylece uzak mesafelerdeki anlamsal bilgi akışı hızlandırılır.

Bu optimizasyonlar sayesinde BigBird, standart 512 token limitine karşın 8 kat daha fazla girdi işleme kapasitesine (4096 token) ulaşabilmektedir. Projemizin temel odağının siber güvenlik ve zafiyet tespiti olması, bu mimarinin ana model olarak seçilmesindeki en büyük motivasyondur. Günümüz LLM uygulamalarında zararlı içeriğin uzun ve zararsız görünen metinlerin ortasına veya en sonuna gizlendiği saldırı senaryolarında [6], 512 token limitli geleneksel modeller zararlı içeriği *truncation* (kesme) işlemi nedeniyle hiç okuyamamaktadır. BigBird'ün kapasitesi, bu gizlenmiş güvenlik ihlallerinin tespiti sorununu doğrudan adreslemiştir.

3.3.1.2 Eğitim İş Akışı ve İleri Seviye Optimizasyonlar. Eğitim süreci; verilerin tabakalı olarak dengeli bölünmesi, dinamik dizi uzunluğu ayarlamaları, ağırlıklı kayıp hesaplaması ve F1-Score odaklı model değerlendirme aşamalarından oluşan sistemli bir yapıda kurgulanmıştır. Şekil 10'te sistenin uçtan uca genel eğitim mimarisi özetlenmektedir. Geliştirilen sistemde standart statik dizi boyutlandırması yerine, GPU bellek israfını engelleyen Dynamic Padding algoritması uygulanmıştır. Dynamic Padding, eğitim veri setindeki tüm metinleri önceden belirlenmiş sabit ve maksimum bir uzunluğa boş (*pad*) token'lar ile tamamlamak yerine, yalnızca o an ağa beslenen veri yığınındaki (batch) en uzun dizinin boyutunu referans alarak doldurma işlemi yapma stratejisidir. BigBird modelinin seyrek dikkat mekanizması, yapısal optimizasyonu gereği verileri 64 token'lık bloklar halinde işlemektedir. Bu kısıt göz önüne alınarak, veri külliyatı işlenirken dinamik olarak belirlenen yığın uzunluğu 64'ün en yakın katına yuvarlanmış ve gereksiz matris hesaplamaları engellenerek eğitim optimize edilmiştir.



Şekil 10: Model Eğitim ve Optimizasyon Akış Şeması

Sisteme entegre edilen bir diğer kritik mühendislik optimizasyonu, *WeightedTrainer* sınıfıdır. Siber güvenlik veri setlerinde güvensiz içeriklerin sayısının genel bağlamlara kıyasla daha az olması, sınıf dengesizliğine yol açmaktadır. Bu durumu önlemek için Çapraz Entropi Kaybı (*Cross-Entropy Loss*) hesaplanırken hedeflenen azınlık sınıfına ağırlık verilmiştir. Bu ceza mekanizması, modelin tehditleri gözden kaçırmasını matematiksel olarak cezalandırarak sistemin genel duyarlılığını korumasına yardımcı olmuştur [1]. Ayrıca paylaşımlı GPU ortamlarında bellek taşımasını önlemek adına model eğitiminde Gradient Checkpointing tekniği kullanılmış

ve bellek tahsisinde yüksek oranda tasarruf sağlanmıştır [3].

3.3.1.3 Model Seçim Kriteri. Nihai model seçiminde temel kriter F1-Score metriği olmuştur. Güvenlik sınıflandırma görevlerinde zararlı içeriği kaçırmamak ne kadar önemliyse, güvenli içerikleri yanlışlıkla zararlı olarak etiketleyip kullanıcı deneyimini bozmamak da o kadar kritiktir. F1-Score, bu iki dengenin (Precision ve Recall) harmonik ortalamasını alarak modelin hem tehditleri yakalama gücünü hem de yanlış alarm vermeme kararlılığını en iyi yansıtan metriktir. Bu nedenle tüm versiyonlarda model ağırlıkları bu kritere göre otonom olarak seçilmiştir.

3.3.1.4 Genel Eğitim Hiperparametreleri. Farklı konfigürasyon versiyonları arası yapılan mimari iyileştirmelerin model davranışı üzerindeki net etkisini şeffaf bir şekilde gözlemleyebilmek adına, modelin temel öğrenme dinamiklerini belirleyen hiperparametreler tüm denemelerde sabit tutulmuştur. Sabitlenen bu kritik eğitim parametreleri Tablo 15'te sunulmuştur.

Hiperparametre	**Değer / Ayar**
`learning_rate`	2e-5
`warmup_ratio`	0.1
`weight_decay`	0.01
`UNSAFE_BOOST`	1.5
`MAX_LENGTH`	4096
`FP16`	True
`gradient_checkpointing`	True

Tablo 15: Tüm Eğitim İterasyonlarında Sabit Tutulan Hiperparametreler

Eğitim sürecinde kullanılan hiperparametrelerin işlevleri belirli akademik standartlara dayanmaktadır. Öğrenme oranı (learning_rate) ince ayar işlemi için literatür standardı olan 2e-5 olarak belirlenmiştir. İlk %10'luk adımda öğrenme oranının kademeli olarak artması (warmup_ratio) sağlanmış ve böylece ani gradyan patlamaları önlenmiştir. Aşırı öğrenmeyi engellemek adına weight_decay parametresi 0.01 olarak uygulanmıştır. Sınıf dengesizliğini çözmek ve modelin tehdit tespit hassasiyetini artırmak için hedef sınıfa UNSAFE_BOOST (1.5) ile ağırlık çarpanı uygulanmıştır. Ayrıca MAX_LENGTH 4096 token ile giriş sınırı belirlenmiş, bellek israfını önlemek için FP16 karışık hassasiyetli eğitim kullanılmış ve gradient_checkpointing ile ara aktivasyonların bellekten silinip geriye yayılımda (backpropagation) yeniden hesaplanması sağlanarak bellek taşması riskleri büyük ölçüde azaltılmıştır.

3.3.1.5 Eğitim Süreci ve Versiyon Geliştirmeleri. Versiyonlar arası değişen konfigürasyon parametreleri Tablo 16'de özetlenmiş olup, eğitim süreci iteratif bir yaklaşımla ilerlemiştir. Tabloda yer alan parametrelerin işlevleri şu şekildedir: *Padding Stratejisi*, girdi dizilerinin uzunluklarının nasıl eşitlendiğini (sabit veya dinamik); *Attention Tipi*, BigBird'ün dikkat mekanizmasının modunu (seyrek veya tam); *Batch Size*, tek seferde işlenen fiziksel veri sayısını; *Gradient Accumulation*, bellek tasarrufu için gradyanların kaç adımda bir güncellendiğini; *Effective Batch Size*, optimizasyon adımındaki gerçek öğrenme hacmini; *Epoch Sayısı*, tüm

veri setinin kaç tur işlendiğini; *Early Stopping*, aşırı öğrenmeyi önlemek için eğitimin erken kesilme toleransını; *Eval Stratejisi* ise modelin değerlendirilme sıklığını ifade etmektedir.

Parametre	**V1**	**V2**	**V3**	**V4 (v3_3_4)**	**V5**
Padding Stratejisi	Sabit 4096	Dynamic	Dynamic	Dynamic	Dynamic
Attention Tipi	block_sparse	block_sparse	original_full	block_sparse	original_full
Batch Size	2	8	8	4	4
Gradient Accumulation	16	4	4	8	8
Effective Batch Size	32	32	32	32	32
Epoch Sayısı	5	5	5	5	3
Early Stopping	Patience=3	Patience=3	Patience=3	Patience=3	Yok
Eval Stratejisi	500 step	500 step	500 step	500 step	Epoch

Tablo 16: Model Versiyonları Arası Değişen Temel Eğitim Parametreleri Özeti

3.3.2 Model Değerlendirme Aşaması

3.3.2.1 Veri Seti Bölünmesi ve Kümelerin Kapsamı. Projede kullanılan veri setinin eğitim, doğrulama ve test kümelerine ayrılması işlemi *scikit-learn* kütüphanesinin `train_test_split` modülü ile gerçekleştirilmiştir [11]. Bölme işlemi sırasında salt rastgelelik yerine özel olarak Stratified Split (Tabakalı Bölme) yöntemi tercih edilmiştir. Siber güvenlik veri setlerinde zararlı içerikler doğası gereği azınlık popülasyonunu temsil etme eğilimindedir. Tabakalı bölme, orijinal veri setindeki hedef sınıf oranlarını matematiksel olarak kilitler ve eğitim alt kümelerinin bu ana dağılımı aynı oranda yansıtmasını sağlar [7]. Dağılım oranları Tablo 17'te gösterilmektedir.

Eğitim sürecinde ayrılan bu doğrulama ve test kümeleri, modelin kendi iç öğrenme, gradyan güncelleme ve hiperparametre optimizasyonu süreçlerinde kullanılmıştır. Bu veri ayrımı, projenin nihai aşamasında modelin gerçek dünya senaryolarına karşı dayanıklılığını ölçmek için tasarlanan dışsal Benchmark testinden tamamen bağımsızdır.

Veri Kümesi	**Oran**	**Kullanım Amacı**
Train	%76.5	Modelin ağ ağırlıklarının gradyan inişi ile güncellenmesi süreci.
Validation	%8.5	Eğitim esnasında hiperparametre optimizasyonu ve overfitting kontrolü.
Test	%15.0	Kendi iç dağılımında eğitim bittikten sonra yapılan model içi değerlendirme.

Tablo 17: Veri Seti Bölünme Oranları ve Kullanım Amacı

3.3.2.2 Değerlendirme Metriklerinin Kapsamı. Bir güvenlik modelinde hata tiplerinin sistem üzerindeki risk maliyetleri birbirinden farklıdır. Kavramları netleştirmek adına, metrik analizlerinde kullanılan ana parametreler şu şekildedir:

- *True Positive (TP):* Gerçekte güvensiz olan ve modelin de başarıyla güvensiz bulduğu içerikler.
- *True Negative (TN):* Gerçekte güvenli olan ve modelin de güvenli olarak bıraktığı içerikler.

- *False Positive (FP - Yanlış Alarm):* Gerçekte güvenli olduğu halde modelin yanılıp güvensiz olarak işaretlediği ve engellediği içerikler. Son kullanıcının deneyimini yavaşlatır ancak teknik bir sistem veya veri zafiyeti yaratmaz.
- *False Negative (FN - Kaçırılan Tehdit):* Gerçekte zararlı olan ancak modelin bunu algılayamayıp sisteme sızmasına izin verdiği durumlardır. Prompt injection veya veri sızıntılarına sebebiyet vereceği için en tehlikeli hata tipidir.

Bu asimetrik risk dağılımı göz önüne alınarak, değerlendirme aşamasında modelin nihai kararını belirlemek üzere şu metrikler merkeze alınmıştır:

- *F1-Score:* Modelin hedef seçim metriğidir. Precision ve Recall değerlerinin harmonik ortalamasıdır:

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN} \tag{5}$$

- *F2-Score:* Güvenlik projelerinde kaçırılan tehdit hatasının maliyeti çok daha yüksek olduğundan, modelin duyarlılık performansına kesinlik performansından iki kat daha fazla ağırlık veren F2-Skoru izlenmiştir:

$$\text{F2-Score} = (1 + 2^2) \times \frac{\text{Precision} \times \text{Recall}}{(2^2 \times \text{Precision}) + \text{Recall}} \tag{6}$$

- *False Negative Rate (FNR):* Toplam gerçek tehditlerin ne kadarının gözden kaçırıldığını gösterir. Doğrudan güvenlik açığını temsil ettiği için sıfıra en yakın olması hedeflenir.

3.3.3 Alınan Sonuçlar ve Performans İncelemesi

Model değerlendirmesi ve iteratif test süreci iki bağımsız koldan yürütülmüştür. Birinci aşamada modelin genel performansını ölçmek amacıyla on binlerce örnekten oluşan standart Test seti kullanılmıştır. İkinci aşamada ise, modelin ekstrem (edge-case) saldırılara karşı dayanıklılığını kanıtlamak için özel tasarlanmış tamamen izole Benchmark veri seti kullanılmıştır.

3.3.3.1 Genel Test Seti Başarımı. Tablo 18'te sunulan yüzdelik oranlar, devasa test setinin bağımsız ölçümleri sonucunda elde edilmiştir. Örneğin V4 modeli için ayrılmış toplam 30.990 adet (yaklaşık 15.500 güvenli ve 15.500 güvensiz) test verisi üzerinde yapılan çıkarımlar sonucunda, doğru sınıflandırılan örneklerin toplam veri sayısına oranı olan genel doğruluk (Accuracy) %96.11 olarak hesaplanmıştır. Sistem performansını asıl belirleyen Unsafe F1-Score değeri ise, modelin hassasiyet (Precision) ve duyarlılık (Recall) dengesinin matematiksel harmonik ortalaması alınarak %96.18 seviyesinde tespit edilmiştir. V1'den V5'e doğru gidildiğinde iterasyon başarılarının artarak V5 sürümünde %98.17 ile en stabil seviyeye ulaştığı görülmektedir.

Versiyon	Accuracy	Unsafe Precision	Unsafe Recall	Unsafe F1-Score
V1	%94.60	%92.25	%97.20	0.9466
V3	%96.38	%95.52	%97.45	0.9647
V4	%96.11	%94.44	%97.99	0.9618
V5	**%98.14**	**%97.96**	**%98.39**	**0.9817**

Tablo 18: Model Versiyonlarına Göre Standart Test Seti Performans Metrikleri

3.3.3.2 Benchmark ile Dayanıklılık Değerlendirmesi. Eğitim setinden tamamen izole edilmiş olan Benchmark veri seti, modelin eğitim sürecinde hiç görmediği zorlayıcı siber güvenlik senaryolarından oluşmaktadır. Bu veri seti genel genellenebilirlikten ziyade, sistemin prompt injection ve diğer zafiyet kategorilerindeki gerçek dünya direncini ölçmek için tasarlanmıştır. Tablo 19'te bu özel setin temel bileşenleri listelenmiştir.

Benchmark Bileşeni	Açıklama ve Amacı
Prompt Injection Örnekleri	Çok uzun ve masum görünen günlük metin bağlamlarına ustaca gizlenmiş zararlı komutlar içerir. Modelin uzun bağlam kapasitesini test eder [6].
Kısa Doğrudan Unsafe	Açık siber tehdit veya manipülasyon içeren kısa metinlerdir. Temel sınıflandırma yetkinliğini sınar.
Türkçe Test Seti	Tamamen Türkçe güvensiz ve güvenli diyalog örnekleridir. Modelin yerel dil üzerindeki ince anlamsal kavrayışını ölçer.
İngilizce Test Seti	Modelin temel alındığı önceden eğitilmiş aklın İngilizce dilinde korunup korunmadığını teyit eder.

Tablo 19: Benchmark Veri Seti Bileşenleri ve Test Amaçları

Tamamı zorlu ve tehlikeli 181 girdiden oluşan güncel Benchmark testinin analizleri sonucunda Tablo 20 elde edilmiştir. Örneğin V4 modelinin satırı incelendiğinde; modelin kendisine verilen 181 zorlu tehdit senaryosunun 169'unu başarılı bir şekilde tehlikeli olarak işaretlediği, ancak 12 tanesine sistemde zararsız diyerek geçiş izni verdiği görülmektedir. Bu 169/181 oranı modelin Benchmark testinde %93.37 başarı oranına ulaştığını göstermektedir. Tablo genelinde en yüksek yakalama oranına V4 modeli ulaşmış olsa da; V5 modeli, kaçırdığı tehdit sayısında V4'e son derece yakın bir profil çizerken genel test setinde gösterdiği çok daha yüksek F1-Score dengesi nedeniyle overfitting yapmayan optimum, nihai model olarak kabul edilmiştir.

Versiyon	Toplam Tehdit	Yakalanan (TP)	Kaçırılan Tehdit (FN)	Benchmark Başarısı (Recall)
V1	181	157	24	%86.74
V2	181	164	17	%90.61
V3	181	160	21	%88.40
V4	**181**	**169**	**12**	**%93.37**
V5	181	156	25	%86.19

Tablo 20: Özel 181 Soruluk Benchmark Veri Seti Üzerinde Tehdit Yakalama Performansı

3.3.3.3 Karşılaşılan Problemler ve Geliştirme Sürecindeki Çözümler. İteratif eğitim süreci boyunca karşılaşılan donanımsal ve tasarımsal darboğazlar, her yeni versiyonda uygulanan optimizasyonlarla giderilmiştir. Modelin 4096 token'a kadar veri alabilme özelliği, ilk aşamalarda (V1) kısa metinlerin dahi statik olarak maksimum uzunluğa doldurulmasına neden olmuş, bu durum matris hesaplamalarını devasa boyutlara çıkararak eğitimin on beş saatin üzerine çıkmasına sebebiyet vermiştir. Bu darboğaz, V2 sürümünde özel bir dinamik doldurma algoritmasıyla çözülmüş, dizi uzunlukları her bir işlem yığınının (batch) kendi içindeki en uzun örneğine sınırlandırılarak eğitim kaliteden ödün verilmeden hızlandırılmıştır. Log incelemelerinde, modelin test setinde anomali yarattığı saptanmış ve kısa dizilerde otomatik olarak seyrek dikkat modundan tam dikkat moduna geçtiği tespit edilmiştir. Ancak tam dikkat mekanizmasına geçiş, V4 sürümünde paylaşımlı GPU donanımında bellek yetersizliği (Out of Memory) çökme hatasına neden olmuştur. Bellek sorunlarını aşmak amacıyla dikkat mekanizması tekrar seyrek dikkate sabitlenmiş, fiziksel işlem yığını 4'e düşürülürken gradyan birikimi 8'e çıkarılarak efektif öğrenme hacmi korunmuştur. Son olarak, veri setine zorlu anomali türlerinin eklenmesi modelin kayıp (loss) dalgalanması yaşamasına neden olmuş ve early stopping mekanizmasını tetikleyerek eğitimin erken sonlanmasına yol açmıştır. Modelin tüm veriyi ve daha geniş bir uzayı kavrayabilmesi için V5 sürümünde erken durdurma mekanizması tamamen kaldırılarak modelin üç tam döngü (epoch) boyunca eğitilmesi sağlanmıştır.

3.3.4 Sonuç

Bu çalışma kapsamında, kurumsal düzeyde Büyük Dil Modelleri'nin (LLM) dış ve iç güvenlik açıklarına karşı korunmasını sağlamak amacıyla BigBird-RoBERTa altyapısına dayanan özelleştirilmiş bir güvenlik bariyeri sistemi geliştirilmiştir. Literatürde sıkça karşılaşılan 512 token limitinin yarattığı zafiyet, seyrek dikkat optimizasyonu ve 4096 token'lık bağlam işleme yeteneği ile kalıcı olarak aşılmıştır. Gerçekleştirilen beş aşamalı iteratif eğitim süreci; sınıf dengesizliklerini çözen ağırlıklandırma metotları, bellek israfını önleyen dinamik doldurma ve gradyan kontrolü gibi ileri seviye mühendislik çözümleriyle desteklenmiştir. Uygulanan bu teknikler neticesinde nihai V5 modeli, standart test ortamında %98.14 doğruluk ve %98.17 F1-Score elde ederken; yalnızca gizlenmiş saldırılardan oluşan zorlayıcı Benchmark testini yüksek bir tehdit yakalama oranı ile geçmeyi başarmıştır. V5 modeli, yanlış alarmlar ile kaçırılan hayati tehditler arasındaki asimetrik riski F1-Score bazlı minimize ederek, canlı sistemlere entegre edilebilecek stabil bir başarım seviyesine ulaşmıştır.

4 Projede Kullanılan Teknolojiler

Bu bölüm, güvenlik sınıflandırma ve veri çevirisi süreçlerinin geliştirilmesi, eğitilmesi ve canlı API ortamında sunulması aşamalarında kullanılan teknoloji yığınını özetlemektedir. Proje, üç kişilik bir mühendislik ekibinin asenkron çalışabilmesini destekleyen modüler bir mimariyle inşa edilmiştir.

4.1 Programlama Dilleri

Projenin tüm modüllerinde ana dil olarak **Python** tercih edilmiştir. Gerçek zamanlı güvenlik sınıflandırması, derin öğrenme model eğitimleri, vLLM API orkestrasyonu, paralel veri çevirisi ve deneysel model testleri gibi süreçlerin tamamı Python ekosistemi üzerinde geliştirilmiştir.

4.2 Kütüphaneler ve Framework'ler

Framework / Araç	Versiyon	Kullanım Alanı
PyTorch	2.0+	Model yükleme ve tensör işlemleri.
Transformers (HF)	4.30+	Önceden eğitilmiş modeller, Trainer API, tokenizasyon.
BigBird	–	4096 token desteği ile uzun prompt işleme.
ModernBERT	–	Modern transformer mimarisi ile güvenlik sınıflandırması (deney).
Longformer	–	4096 token ile uzun doküman sınıflandırması (deney).
XLM-RoBERTa	–	100+ dil desteği, Sliding Window (deney).
BERT Turkish	–	dbmdz/bert-base üzerinden Türkçe sınıflandırma, 512 token (deney).
Datasets (HF)	2.14+	Veri seti yönetimi, ön işleme, HF Hub entegrasyonu.
Evaluate (HF)	0.4+	Accuracy, F1-score, ROC-AUC hesabı.
scikit-learn	1.3+	train_test_split, karmaşıklık matrisi, sınıflandırma raporları.

Tablo 21: Derin Öğrenme ve Makine Öğrenmesi Kütüphaneleri ve Kullanım Alanları

Veri İşleme, Görselleştirme ve Ağ İletişimi

- **Veri İşleme:** pandas (DataFrame analizleri), numpy (sayısal hesaplamalar), pyarrow (veri serileştirme).
- **API ve Ağ:** requests (Senkron HTTP POST işlemleri), aiohttp (Asenkron paralel API istekleri).
- **Görsel İşleme:** PIL/Pillow (Görüntü tabanlı sınıflandırma ön işlemleri).

Paralelizasyon ve Eşzamanlılık (Multi-tasking)

Ağ darboğazlarını ve eğitim sürelerini optimize etmek için çoklu görev araçları kullanılmıştır:

- **concurrent.futures & threading:** Paralel API istekleri ve batch işleme.
- **asyncio:** vLLM API'sine yönelik asenkron çağrılar.

- **vLLM:** Yüksek verimli paralel LLM çıkarım (inference) sunucusu.

Yardımcı Araçlar (Utilities)

- **Konfigürasyon:** python-dotenv (.env yönetimi), pyyaml (YAML konfigürasyonları).
- **İşlem Takibi & Loglama:** logging (detaylı hata takibi), tqdm (eğitim/çıkarım ilerleme çubukları).
- **Veri & Model Güvenliği:** safetensors (güvenli model serileştirme), langdetect (otomatik dil tespiti).

4.3 Geliştirme Ortamları ve Araçlar

- **IDE & Geliştirme:** Jupyter Notebook (etkileşimli analiz), Visual Studio Code (geliştirme, hata ayıklama, Git entegrasyonu), Python REPL/Scriptleri.
- **Paket Yönetimi:** Python venv (sanal ortam izolasyonu), pip, requirements.txt ve pyproject.toml.
- **Model Kayıt ve İzleme:** HuggingFace Model Hub (önceden eğitilmiş modeller), .pt/.bin/.safetensors formatlarında yerel ağırlık kayıtları (checkpointing), stdout yönlendirmeli nohup log dosyaları.
- **Veri Yönetimi:** JSON/JSONL formatında temel veri depolama ve HuggingFace Datasets tabanlı versiyon kontrolü.

5 Sonuç ve Gelecek Çalışmalar

5.1 Genel Değerlendirme

Bu proje kapsamında, HAVELSAN bünyesinde geliştirilen Main büyük dil modeli için Türkçe ve İngilizce dillerinde yüksek performans gösteren, güvenilir ve özelleştirilebilir bir guardrail sistemi başarıyla geliştirilmiştir. Projenin başlangıcında belirlenen temel hedef fazlasıyla aşılmıştır. Model eğitim aşamalarında detaylandırıldığı üzere; optimize edilen BigBird V5 modeli standart test veri setinde %98.14 doğruluğa ulaşırken, ModernBERT mimarisi %98.63 doğruluk elde ederek her iki mimaride de %98'in üzerinde bir başarım sergilemiştir.

Mevcut açık kaynaklı çözümlerin Türkçe dilindeki yetersizlikleri, projeye özel tasarlanan titiz bir veri hazırlama pipeline ile giderilmiştir. Çeşitli açık kaynaklı veri setlerinin anlam kayması yaşanmadan Türkçeye çevrilmesi, Qwen tabanlı kalite kontrol filtrelemesi ve vLLM destekli sentetik veri üretimi, modelin öğrenme uzayını son derece zenginleştirmiştir.

Model mimarisi tarafında ise 512 token limiti aşılmış; BigBird (4096 token) ve ModernBERT (8192 token) gibi uzun bağlam işleyebilen mimariler kullanılarak, gerçek dünya senaryolarında en büyük tehditlerden biri olan uzun metinlere gizlenmiş "Prompt Injection" saldırılarına karşı yüksek bir direnç elde edilmiştir. Bu direnç, özellikle sadece zararlı (unsafe) içeriklerin uzun bağlamlara gizlenmesiyle oluşturulan "Benchmark Veri Seti" testlerinde kanıtlanmıştır. Tablo 2.4'te de görüldüğü üzere, BigBird V4 modeli %93.09, V5 modeli ise %92.02 Benchmark başarı oranına ulaşarak False Negative sayılarını sıfıra yaklaştırmıştır.

Elde edilen bu sonuçlar, geliştirilen yerli çözümün kurumsal düzeyde bir LLM entegrasyonu için üretime hazır, stabil ve yüksek güvenilirlikli olduğunu kanıtlamaktadır.

5.2 Gelecek Çalışmalar

Proje mevcut haliyle temel başarı kriterlerini fazlasıyla karşılamış olsa da, guardrail sisteminin sürekli gelişen LLM tehditlerine karşı güncel tutulması ve performansının daha da artırılması için aşağıdaki adımlar gelecek çalışmalar olarak planlanmaktadır:

- **Sürekli Öğrenme ve Aktif Veri Toplama:** Model canlı ortama alındıktan sonra, kullanıcıların Main LLM ile olan gerçek etkileşimlerinden elde edilecek verilerle bir geri bildirim döngüsü (RLHF veya DPO) kurulması hedeflenmektedir. Canlı sistemdeki yoğun istek trafiğini kesintisiz ve hatasız yönetebilmek adına sisteme Load Balancer mimarisi entegre edilecektir. Bu sayede modelin zamanla ortaya çıkabilecek yeni "Jailbreak" taktiklerine karşı darboğaz yaşanmadan, asenkron ve dinamik olarak eğitilmesi sağlanacaktır.

- **Çıkarım Hızlandırma ve Optimizasyon:** Gerçek zamanlı sistemlerdeki gecikmeyi (latency) minimuma indirmek amacıyla, elde edilen nihai model ağırlıklarının Quantization (Örn: INT8, AWQ) veya TensorRT gibi yöntemlerle optimize edilmesi planlanmaktadır. Buna ek olarak, modelin donanım kısıtlamalarından bağımsız, en yüksek hızda ve esneklikte çalışabilmesi için ONNX (Open Neural Network Exchange) formatına dönüştürülmesi ve çıkarım süreçlerinin ONNX Runtime üzerinden sunulması hedeflenmektedir. Bu geliştirmelerle GPU kaynak kullanımının daha da verimli hale getirilmesi sağlanacaktır.

- **Çok Sınıflı Sınıflandırma ve Detaylı Analitik:** Mevcut ikili (Safe/Unsafe) sınıflandırma yapısının, tehdit türlerine göre (Örn: Siber Saldırı, Nefret Söylemi, Dezenformasyon vb.) çok sınıflı bir yapıya genişletilmesi. Bu geliştirme, güvenlik ihlallerinin türüne göre farklı aksiyonların veya özel hata mesajlarının tetiklenmesine olanak tanıyacaktır.

Kaynakça

[1] Aurelio, Y. S., de Almeida, G. M., de Castro, C. L., & Braga, A. P. (2019). Learning from imbalanced data sets with weighted cross-entropy function. *Neural Processing Letters*, 50(2), 1937–1949.

[2] I. Beltagy, M. E. Peters, and A. Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

[3] Chen, T., Xu, B., Zhang, C., & Guestrin, C. (2016). Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*.

[4] A. Conneau et al. Unsupervised cross-lingual representation learning at scale. In *Proc. ACL*, 2020, pp. 8440–8451.

[5] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.

[6] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*.

[7] Kuhn, M., & Johnson, K. (2013). *Applied predictive modeling* (Vol. 26). New York: Springer.

[8] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár. Focal loss for dense object detection. In *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2017, pp. 2980–2988.

[9] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *Proc. ICLR*, 2019.

[10] P. Micikevicius et al. Mixed precision training. In *Proc. ICLR*, 2018.

[11] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[12] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna. Rethinking the inception architecture for computer vision. In *Proc. CVPR*, 2016, pp. 2818–2826.

[13] A. Vaswani et al. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[14] Zaheer, M., Guruganesh, G., Dubey, K. A., Ainslie, J., Alberti, C., Ontanon, D., ... & Ahmed, A. (2020). Big Bird: Transformers for longer sequences. *Advances in Neural Information Processing Systems*, 33, 17283–17297.